

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎



数据本身不会产生价值
只有经过分析才有可能产生价值

2.7 海量数据的交互式分析工具Dremel

MapReduce

随着（移动）互联网的发展，MapReduce作为一种面向**批处理**的框架，在很多应用领域已经不太适用。对此，出现了两种应对思路：

1. **改造**MapReduce，使其除了能进行批处理之外，还能进行其他类型的数据处理，例如**流处理**。
2. **抛开**MapReduce，重新设计新的架构——**Dremel**。

2.7 海量数据的交互式分析工具Dremel

- ▶ 2.7.1 产生背景
- 2.7.2 数据模型
- 2.7.3 嵌套式的列存储
- 2.7.4 查询语言与执行
- 2.7.5 性能分析
- 2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 产生背景

MapReduce

优点：便携

缺点：效率低、延迟高

设想：数据分析师在编写完一段代码之后，想立刻验证下这段代码是否可以有效地从海量的数据集中提取出有效的特征，于是他运行这段代码。

如果是利用MapReduce，由于数据量巨大，可能需要等待几个小时或者更长时间。如果发现代码的算法有问题，无法有效提取数据特征，则需要修改后重新运行。这个过程可能会反复很多次，总的耗时可能多达数天。



2.7 海量数据的交互式分析工具Dremel

● 产生背景

类似这样的**数据探索** (Data Exploration) 的应用在实际中非常普遍，很多时候用户更期望的是**实时的数据交互**，就像传统的SQL查询一样。

用户希望提交完自己的请求之后，在一个相对可以接受的合理时间内，系统就会返回结果，而不是像MapReduce这样，需要耗费很长的时间。

随着对**实时数据交互**的应用需求越来越强烈，Google在借鉴了搜索引擎和并行数据库之后，开发出了**实时的交互式查询系统Dremel**。



2.7 海量数据的交互式分析工具Dremel

- **Dremel支持的典型应用包括：**

Web文档的分析

Android市场的应用安装数据的跟踪

Google产品的错误报告

Google图书的光学字符识别

欺诈信息的分析

Google地图的调试

Bigtable实例上的tablet迁移

Google分布式构建系统的测试结果分析

磁盘I/O信息的统计

Google数据中心上运行任务的资源监控

Google代码库的符号和依赖关系分析

当进行负载均衡、节点故障恢复或者存储资源调整等情况时，就需要将 tablet 从一个节点迁移到另一个节点。在这个过程中，需要了解 tablet 的数据分布、访问模式、数据热度、元数据大小等，再依据这些信息分配存储资源、规划迁移路径等。完成迁移后，还要对比前后数据的一致性、完整性，完成迁移验证工作。

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

► 2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 两方面的技术支持

Google的数据平台需要满足**通用性**，即**不同平台**之间能够很好地实现数据的**交互**处理。此时，想要实现实时交互查询，就需要两方面的技术支持：

两方面的
技术支持

一方面：统一的存储平台

能够实现高效的数据存储，Dremel使用的底层数据存储平台是**GFS**。

另一方面：统一的数据存储格式

存储的数据才可以被不同的平台所使用。

实现统一的格式要考虑：**数据模型**和模型的**具体实现**

2.7 海量数据的交互式分析工具Dremel

- 面向记录和面向列的存储

关系数据库中，用关系模型对其存储的数据进行建模，统一的模型一定程度上简化了数据的存储和查询。但现实中，很多数据之间并没有严格的关系，而是一种松散和扁平的结构。

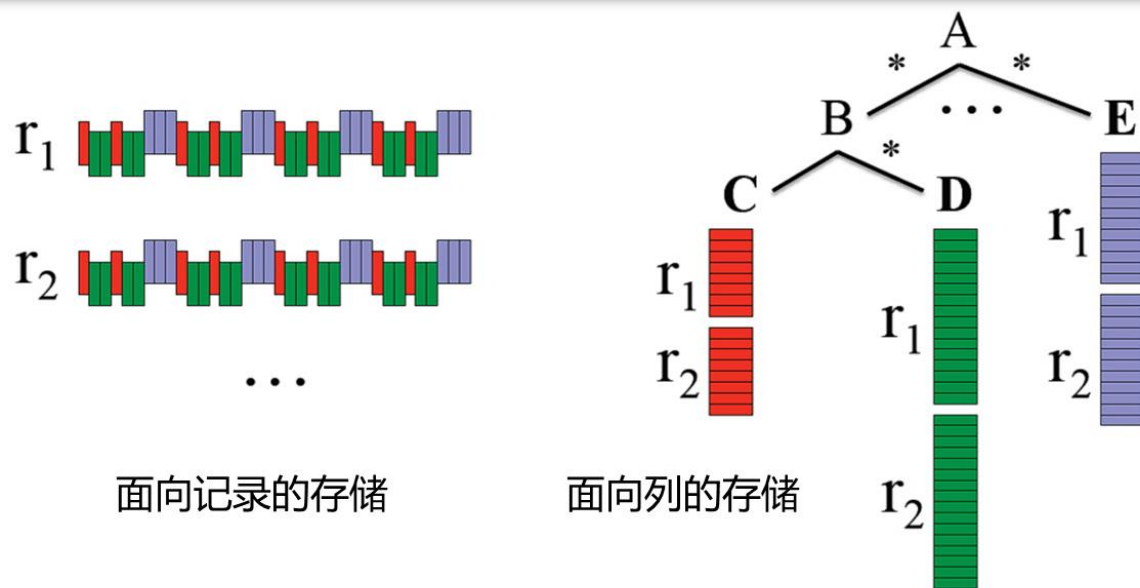
关系数据库的数据存储方式一般有两种：行存储和列存储。

2.7 海量数据的交互式分析工具Dremel

● 面向记录和面向列的存储

行存储：也叫元组存储或面向记录的存储。面向记录，以**行**为单位进行存储。

列存储：以**属性**为单位，每次存储一个属性，在应用时再将需要的属性重新组装成原始的记录。



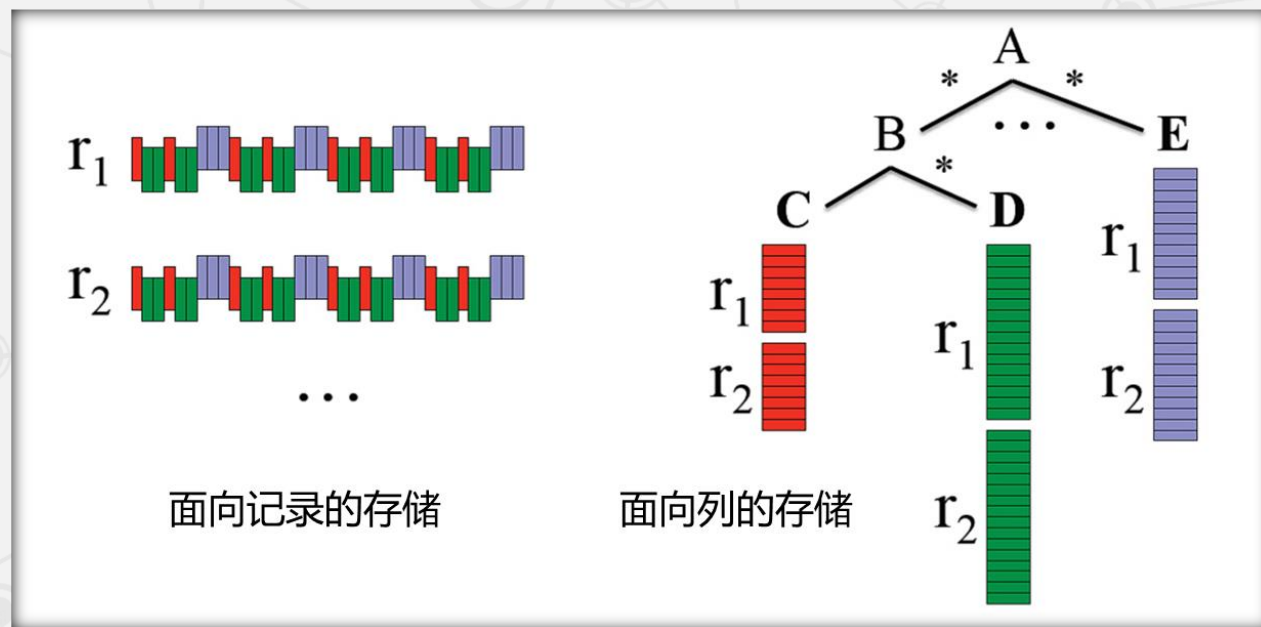
Google发现，**嵌套数据模型**（允许数据以包含其他结构的方式存在）很适合Google的数据存储。

2.7 海量数据的交互式分析工具Dremel

● 面向记录和面向列的存储

Dremel 的嵌套数据模型可以看作是一个用于表示复杂数据结构的树形结构

Google的Dremel是第一个在嵌套数据模型基础上实现列存储的系统。列存储的主要好处在于：



好处一：

处理时，只需要使用涉及到的列数据。

好处二：

更利于数据的压缩。

2.7 海量数据的交互式分析工具Dremel

- 下式为嵌套模型的形式化定义

$$\tau = \text{dom} \mid \langle A_1 : \tau[*|?], \dots, A_n : \tau[*|?] \rangle$$

DATA_TYPE = SCALAR_TYPE | STRUCT<field1: DATA_TYPE, field2: DATA_TYPE, ...> |
ARRAY<DATA_TYPE>

τ 可以为原子类型 (Atomic Type) 也可以是记录类型 (Record Type)

原子类型允许的取值类型包括整型、浮点型、字符串等。记录类型则可以包含多个域。

A_i 代表该记录型数据的名称

记录型数据包括三种类型：**必须的 (Required)**、**可重复的 (Repeated)** 以及**可选的 (Optional)**。其中，Required的类型必须出现，且仅能出现一次。

2.5 分布式存储系统Megastore

● 照片共享服务数据模型实例

```
CREATE SCHEMA PhotoApp;

CREATE TABLE User {
  required int64 user_id;
  required string name;
} PRIMARY KEY(user_id), ENTITY GROUP ROOT;

CREATE TABLE Photo {
  required int64 user_id;
  required int32 photo_id;
  required int64 time;
  required string full_url;
  optional string thumbnail_url;
  repeated string tag;
} PRIMARY KEY(user_id, photo_id),
  IN TABLE User,
  ENTITY GROUP KEY(user_id) REFERENCES User;

CREATE LOCAL INDEX PhotosByTime
  ON Photo(user_id, time);

CREATE GLOBAL INDEX PhotosByTag
  ON Photo(tag) STORING (thumbnail_url);
```

对比 Megastore

这些属性可以被设置成必需的 (required)、可选的 (optional) 或者可重复的 (repeated, 即允许单个属性上有多个值)。

2.7 海量数据的交互式分析工具Dremel

• 记录类型文档的实例

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

文档的模式 (Schema) 定义

```
DocId: 20      r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

符合该模式的两条记录

r₁和r₂中的属性是通过完整的路径来表示的，比如Name.Language.Code。

2.7 海量数据的交互式分析工具Dremel

● 记录类型文档的实例

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20      r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

这种嵌套式数据模型定义方式跟具体的语言和平台无关。

利用该数据模型，可以使用Java语言，也可以使用C++语言来处理数据，甚至可以用Java编写的MapReduce程序直接处理C++语言产生的数据集。这种跨平台的优良特性正是Google所需要的。

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

► 2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 嵌套结构的模式

关系型数据库中采用列存储的好处是，不同列中相同位置的数据必然属于原数据库中的同一行，因此，可以直接将每一列的值按顺序排列下来，不用引入其他概念，也不会丢失数据信息。

结构关系：但针对Google的这种嵌套式数据结构的列存储，数据之间的关系比关系数据库要复杂。存储后的数据本身反映不出任何结构上的信息，因此，存储中除了记录值，还要记录结构。

数据重组：另外，所有的列存储在应用时往往要涉及多个列，如何按照正确的顺序快速地进行数据重组，也是列存储需要解决的问题。

2.7 海量数据的交互式分析工具Dremel

- 嵌套结构的模式

Dremel从数据结构的表示等方面解决了上述问题。

1. 数据结构的无损表示

2. 高效的数据编码

3. 数据重组

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

r₁

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

r₂

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

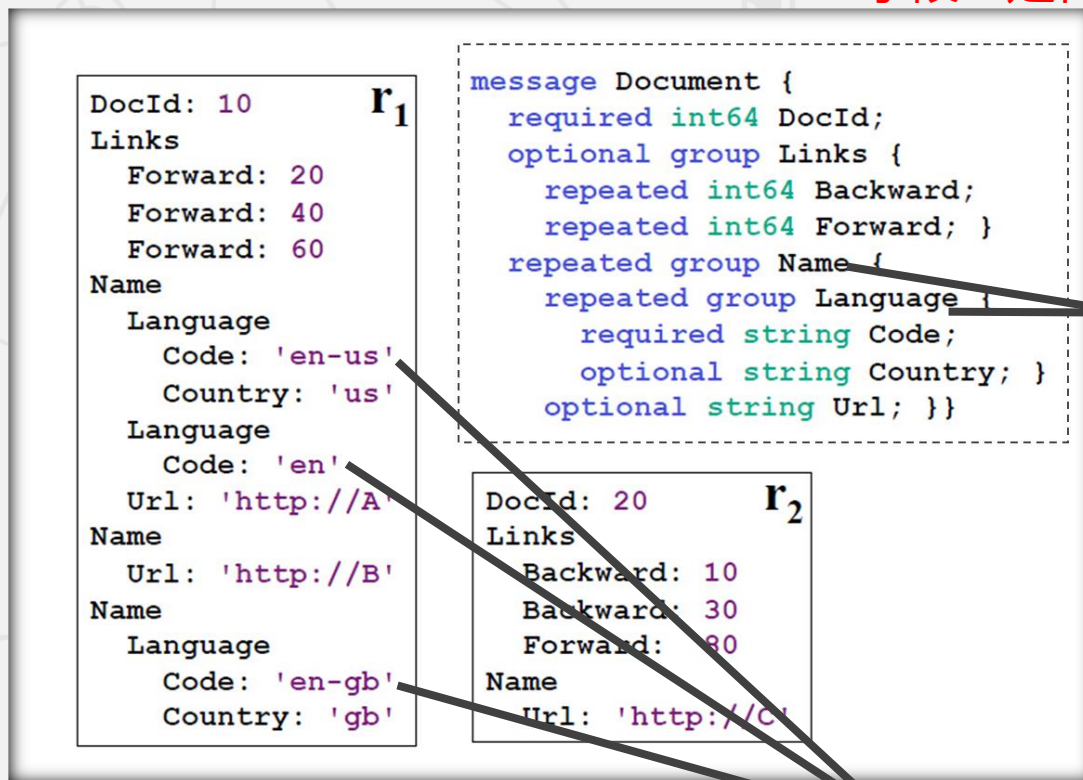
为了准确地反映嵌套结构，Dremel定义了两个变量：**重复深度r** (repeated level) 和**定义深度d** (definition level)。

在嵌套式的数据模型中，对于某个值，例如'**en-us**' 和 '**en**'，如果仅仅是单纯地记录，根本无法判断这两个值对应的是r₁中的哪个位置，因为**Name.Language.Code**出现了**3**次。

2.7 海量数据的交互式分析工具Dremel

1. 数据结构的无损表示

重复深度记录的是该列的值相关的“可重复字段”是在哪个级别上重复的。



对照模式定义，可重复
(repeated) 字段有两个：
Name和Language

Code的可重复深度的取值只
可能为0、1、2，其中，0表
示一个新记录的开始。

Name.Language.Code出现了三次，值分别是“en-us”、“en”、“en-gb”。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20      r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

当“en”出现的时候，Language出现了第二次。因为Language在Name.Language.Code路径中的位置排在第2，所以，“en”的重复深度 $r=2$ 。

沿着 r_1 从上往下读，当读取到“en-us”时，Name和Language都是第一次出现（尚未重复），所以，“en-us”的重复深度 $r=0$ 。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

r₁

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated_group Name {
    repeated_group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

r₂

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

因此，r₁中Code的重复深度分别为0、2、1。

简单来说，重复深度记录的是该列的值是在哪个级别上重复的。

当读取到“en-gb”时，Name重复了，但在此Name后的Language只出现过一次（没有重复），而Name在Name.Language.Code路径中的位置排在第1。因此，“en-gb”的重复深度r=1。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

DocId: 10 **r₁**

Links
Forward: 20
Forward: 40
Forward: 60

Name
Language
Code: 'en-us'
Country: 'us'

Language
Code: 'en'

Url: 'http://A'

Name
Url: 'http://B'

Name
Language
Code: 'en-gb'
Country: 'gb'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

DocId: 20 **r₂**

Links
Backward: 10
Backward: 30
Forward: 80

Name
Url: 'http://C'

Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

由于Language字段中的Code是必需的 (required)，所以它的缺失意味着Language也没有定义。

注意，第二个Name在r₁中没有包含任何Code值。为了确定“en-gb”出现在第三个Name中，系统会添加一个NULL值在“en”和“en-gb”之间。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20      r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

- ✓ 从上述过程可以看出，Dremel的重复深度跟树的深度概念不一样。树每向下延伸一层，深度就增加一层。
- ✓ 但是Dremel中重复深度的定义并不考虑非repeated类型的字段（只考虑repeated类型字段），因为required和optional类型的值不可能重复。
- ✓ 所以，只考虑repeated类型字段出现的情况就可以完整表达出嵌套结构中字段的重复情况。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

- 重复深度r主要关注的是可重复（**repeated**）类型
- 而定义深度d同时关注可重复（**repeated**）类型和可选（**optional**）类型。

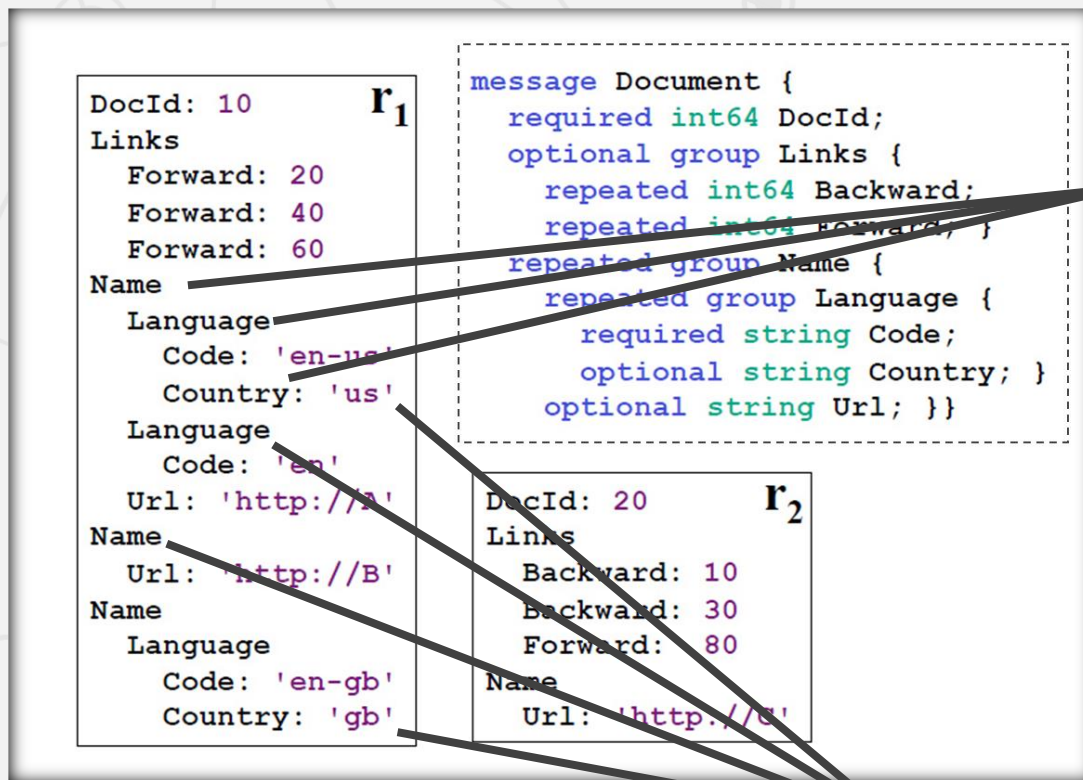
定义深度d表示“值的路径中有多少可以不被定义（即非required类型，包括repeated和optional）的字段实际是有定义的”。

以Name.Language.Country路径为例，Name、Language和Country三个字段均属于“**可有可无型**”，所以其定义深度的可能取值为0、1、2、3。

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```


2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示



对于 “us” , Name、Language、Country都是有定义的, 所以其定义深度为 $d=3$.

Name.Language.Country在r₁中一共有4个定义, 值分别是 “us” 、 “NULL” 、 “NULL” 、 “gb” 。

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

r₁

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

r₂

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

对于“us”，Name、Language、Country都是有定义的，所以其定义深度为d=3.

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

同理，第一个NULL的d=2，第二个NULL的d=1，“gb”的d=3.

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

DocId: 10 **r₁**

Links
Forward: 20
Forward: 40
Forward: 60

Name
Language
Code: 'en-us'
Country: 'us'
Language
Code: 'en'
Url: 'http://A'

Name
Url: 'http://B'

Name
Language
Code: 'en-gb'
Country: 'gb'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

DocId: 20 **r₂**

Links
Backward: 10
Backward: 30
Forward: 80

Name
Url: 'http://C'

DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

2.7 海量数据的交互式分析工具Dremel

1. 数据结构的无损表示

带有重复深度和定义深度的 r_1 与 r_2 的列存储

DocId

value	r	d
10	0	0
20	0	0

Name.Url

value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward

value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward

value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

每一列最终会被存储为块 (Block) 的集合，每个块包含重复深度 r 和定义深度 d ，且包含字段值。

NULLs没有明确存储，因为他们可以根据定义深度确定：任何定义深度小于可重复和可选字段数量之和，就意味着这是一个NULL。

2.7 海量数据的交互式分析工具Dremel

1. 数据结构的无损表示

DocId: 10 **r₁**
Links
Forward: 20

```
message Document {  
  required int64 DocId;  
  optional group Links {
```

DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

必须 (**required**) 字段的值不需要定义深度。

类似地, 重复深度**r**只在必要时存储; 比如, 定义深度**d=0**意味着重复深度**r=0**, 所以后者可以省略。

2.7 海量数据的交互式分析工具Dremel

1. 数据结构的无损表示

DocId: 11

Links

Backward: 10

Backward: 30

Name

Identity

Role: 'teacher'

Jender: 'female'

Identity

Role: 'student'

Number: '2023112009'

Name

Number: '2021225883'

Name

Identity:

Role: 'student'

Jender: 'male'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
  }  
  repeated group Name {  
    repeated group Identity {  
      required string Role;  
      optional string Jender;  
    }  
    optional string Number;  
  }  
}
```

DocId

value	r	d

Name.Number

value	r	d

Links.Backward

value	r	d

Name.Identity.Role

value	r	d

Name.Identity.Jender

value	r	d

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

DocId: 11

Links

Backward: 10

Backward: 30

Name

Identity

Role: 'teacher'

Jender: 'female'

Identity

Role: 'student'

Number: '2023112009'

Name

Number: '2021225883'

Name

Identity:

Role: 'student'

Jender: 'male'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward; }  
  repeated group Name {  
    repeated group Identity {  
      required string Role  
      optional string Jender; }  
    optional string Number; } }
```

DocId

value	r	d
11	0	0

Name.Number

value	r	d
2023112009	0	2
2021225883	1	2
Null	1	1

2.7 海量数据的交互式分析工具Dremel

1. 数据结构的无损表示

DocId: 11

Links

Backward: 10

Backward: 30

Name

Identity

Role: 'teacher'

Jender: 'female'

Identity

Role: 'student'

Number: '2023112009'

Name

Number: '2021225883'

Name

Identity:

Role: 'student'

Jender: 'male'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward; }  
  repeated group Name {  
    repeated group Identity {  
      required string Role  
      optional string Jender; }  
    optional string Number; } }
```

Links.Backward

value	r	d
10	0	2
30	1	2

Name.Identity.Role

value	r	d
teacher	0	2
student	2	2
Null	1	1
student	1	2

2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

DocId: 11

Links

Backward: 10

Backward: 30

Name

Identity

Role: 'teacher'

Jender: 'female'

Identity

Role: 'student'

Number: '2023112009'

Name

Number: '2021225883'

Name

Identity:

Role: 'student'

Jender: 'male'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward; }  
  repeated group Name {  
    repeated group Identity {  
      required string Role  
      optional string Jender; }  
    optional string Number; } }
```

Name.Identity.Jender

value	r	d
female	0	3
Null	2	2
Null	1	1
male	1	3

2.7 海量数据的交互式分析工具Dremel

● 2. 高效的数据编码

Procedure 定义了一个函数 DissectRecord, 该函数有3个输入变量, 包括解码器decoder, writer和重复深度repetitionLevel.

然后, 定义了重复深度和定义深度 (r, d)

seenFields用于判断某个字段是否已经处理。 seenFields的值会影响重复深度。

计算重复和定义深度的基础算法如下图所示。

```
1 procedure DissectRecord(RecordDecoder decoder,  
2                          FieldWriter writer, int repetitionLevel):  
3   Add current repetitionLevel and definition level to writer  
4   seenFields = {} // empty set of integers  
5   while decoder has more field values  
6     FieldWriter chWriter =  
7       child of writer for field read by decoder  
8     int chRepetitionLevel = repetitionLevel  
9     if set seenFields contains field ID of chWriter  
10      chRepetitionLevel = tree depth of chWriter  
11    else  
12      Add field ID of chWriter to seenFields  
13    end if  
14    if chWriter corresponds to an atomic field  
15      Write value of current field read by decoder  
16      using chWriter at chRepetitionLevel  
17    else  
18      DissectRecord(new RecordDecoder for nested record  
19        read by decoder, chWriter, chRepetitionLevel)  
20    end if  
21  end while  
22 end procedure
```

计算重复和定义深度的基础算法

2.7 海量数据的交互式分析工具Dremel

• 2. 高效的数据编码

算法遍历记录结构 (decoder) , 即主循环while decoder就是挨个读取解码器decoder中的数据, 直到decoder中没有值。

然后计算每个列值的深度 (r, d) , 即使是NULL时也不例外。

如果chWriter读取的字段是一个原子字段, 则将其记录下来。

如果不是原子字段, 则递归调用本函数。

```
1 procedure DissectRecord(RecordDecoder decoder,
2     FieldWriter writer, int repetitionLevel):
3     Add current repetitionLevel and definition level to writer
4     seenFields = {} // empty set of integers
5     while decoder has more field values
6         FieldWriter chWriter =
7             child of writer for field read by decoder
8         int chRepetitionLevel = repetitionLevel
9         if set seenFields contains field ID of chWriter
10             chRepetitionLevel = tree depth of chWriter
11         else
12             Add field ID of chWriter to seenFields
13         end if
14         if chWriter corresponds to an atomic field
15             Write value of current field read by decoder
16             using chWriter at chRepetitionLevel
17         else
18             DissectRecord(new RecordDecoder for nested record
19                 read by decoder, chWriter, chRepetitionLevel)
20         end if
21     end while
22 end procedure
```

计算重复和定义深度的基础算法

2.7 海量数据的交互式分析工具Dremel

● 2. 高效的数据编码

在Google, 经常会出现模式包含成千上万的字段, 却仅有几百个在记录中被使用的情况。

因此, 需要尽可能廉价地处理缺失字段。

```
1 procedure DissectRecord(RecordDecoder decoder,
2     FieldWriter writer, int repetitionLevel):
3     Add current repetitionLevel and definition level to writer
4     seenFields = {} // empty set of integers
5     while decoder has more field values
6         FieldWriter chWriter =
7             child of writer for field read by decoder
8         int chRepetitionLevel = repetitionLevel
9         if set seenFields contains field ID of chWriter
10             chRepetitionLevel = tree depth of chWriter
11         else
12             Add field ID of chWriter to seenFields
13         end if
14         if chWriter corresponds to an atomic field
15             Write value of current field read by decoder
16             using chWriter at chRepetitionLevel
17         else
18             DissectRecord(new RecordDecoder for nested record
19                 read by decoder, chWriter, chRepetitionLevel)
20         end if
21     end while
22 end procedure
```

计算重复和定义深度的基础算法

2.7 海量数据的交互式分析工具Dremel

• 2. 高效的数据编码

Dremel利用右图算法计算重复和定义深度，创建一个树状结构。

树的节点为字段的writer，它的结构与模式中的字段层级匹配。

核心的想法是只在字段writer有自己的数据时执行更新，非必要时不尝试往下传递父节点状态。

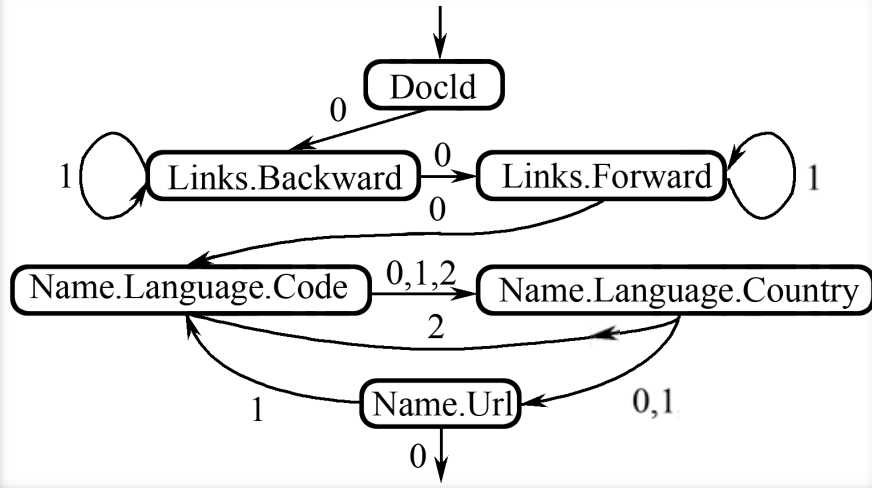
子节点writer继承父节点的深度值。

当任意值被添加时，子writer将深度值同步到父节点。

复盘一下

```
1 procedure DissectRecord(RecordDecoder decoder,  
2     FieldWriter writer, int repetitionLevel):  
3     Add current repetitionLevel and definition level to writer  
4     seenFields = {} // empty set of integers  
5     while decoder has more field values  
6         FieldWriter chWriter =  
7             child of writer for field read by decoder  
8         int chRepetitionLevel = repetitionLevel  
9         if set seenFields contains field ID of chWriter  
10            chRepetitionLevel = tree depth of chWriter  
11        else  
12            Add field ID of chWriter to seenFields  
13        end if  
14        if chWriter corresponds to an atomic field  
15            Write value of current field read by decoder  
16            using chWriter at chRepetitionLevel  
17        else  
18            DissectRecord(new RecordDecoder for nested record  
19                read by decoder, chWriter, chRepetitionLevel)  
20        end if  
21    end while  
22 end procedure
```

计算重复和定义深度的基础算法



```

1 procedure DissectRecord(RecordDecoder decoder,
2   FieldWriter writer, int repetitionLevel):
3   Add current repetitionLevel and definition level to writer
4   seenFields = {} // empty set of integers
5   while decoder has more field values
6     FieldWriter chWriter =
7       child of writer for field read by decoder
8     int chRepetitionLevel = repetitionLevel
9     if set seenFields contains field ID of chWriter
10      chRepetitionLevel = tree depth of chWriter
11    else
12      Add field ID of chWriter to seenFields
13    end if
14    if chWriter corresponds to an atomic field
15      Write value of current field read by decoder
16      using chWriter at chRepetitionLevel
17    else
18      DissectRecord(new RecordDecoder for nested record
19        read by decoder, chWriter, chRepetitionLevel)
20    end if
21  end while
22 end procedure

```

数据的重组

数据编码

DocId: 10 **r₁**

Links

Forward: 20

Forward: 40

Forward: 60

Name

Language

Code: 'en-us'

Country: 'us'

Language

Code: 'en'

Url: 'http://A'

Name

Url: 'http://B'

Name

Language

Code: 'en-gb'

Country: 'gb'

数据的无损表示

DocId	Name.Url	Links.Forward	Links.Backward
value r d	value r d	value r d	value r d
10 0 0	http://A 0 2	20 0 2	NULL 0 1
20 0 0	http://B 1 2	40 1 2	10 0 2
	NULL 1 1	60 1 2	30 1 2
	http://C 0 2	80 0 2	

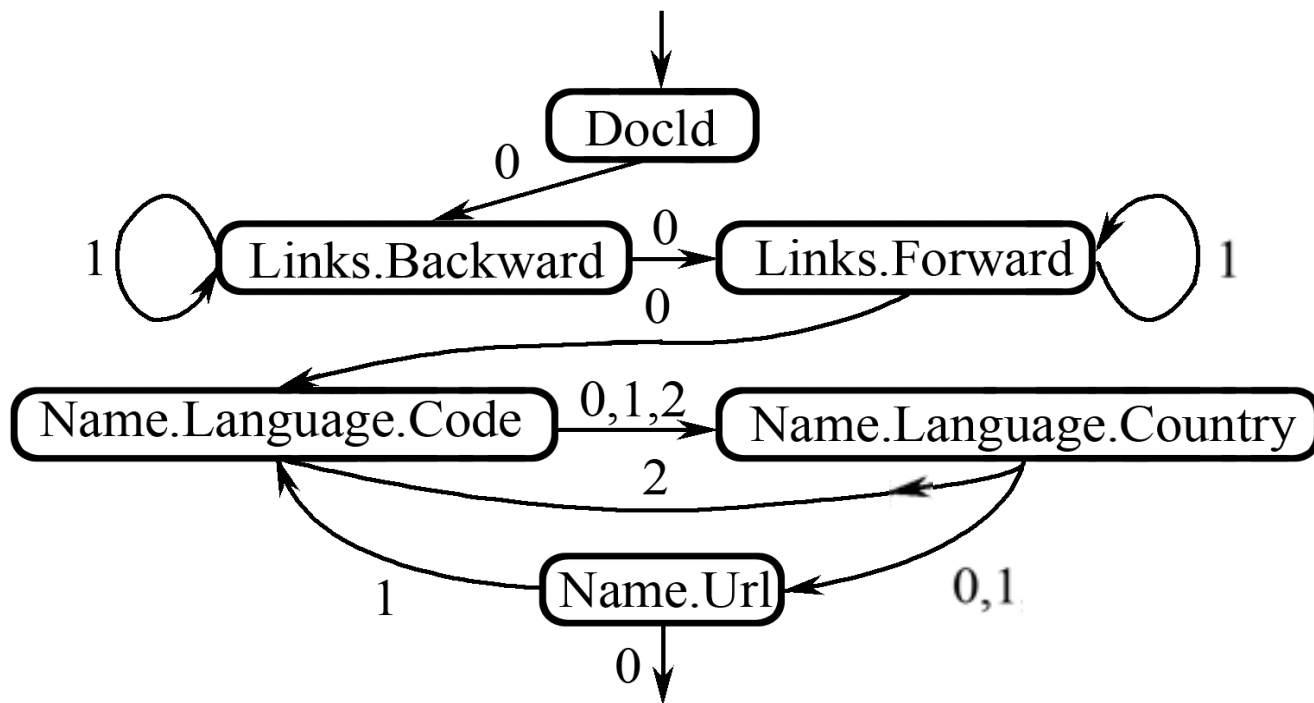
Name.Language.Code	Name.Language.Country
value r d	value r d
en-us 0 2	us 0 3
en 2 2	NULL 2 2
NULL 1 1	NULL 1 1
en-gb 1 2	gb 1 3
NULL 0 1	NULL 0 1

2.7 海量数据的交互式分析工具Dremel

● 3.数据重组

Dremel数据重组方法的核心思想：为每个字段创建一个有限状态机（FSM），读取字段值和重复深度，然后顺序地将值添加到输出结果上。

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```



2.7 海量数据的交互式分析工具Dr

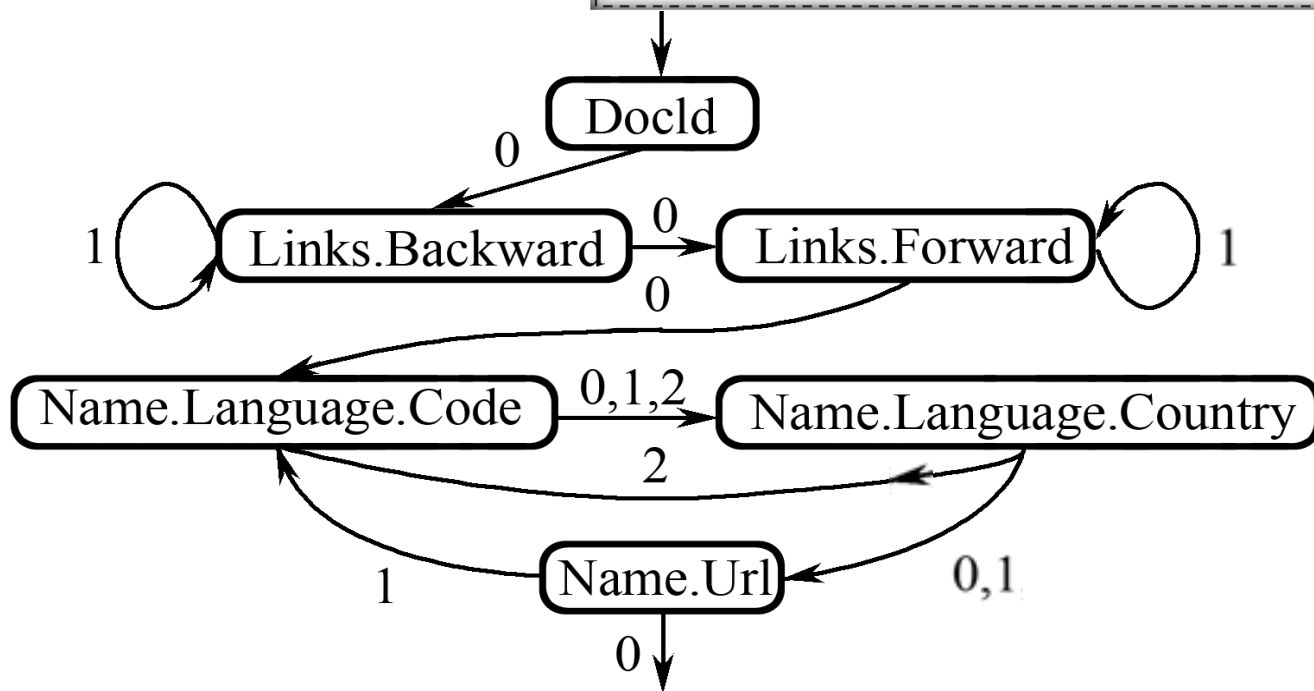
● 3.数据重组

一个字段的FSM状态对应这个字段的reader。

重复深度驱动状态变迁。

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; }};
```

DocId: 10 **r₁**
Links
 Forward: 20
 Forward: 40
 Forward: 60
Name
 Language
 Code: 'en-us'
 Country: 'us'
 Language
 Code: 'en'
 Url: 'http://A'
Name
 Url: 'http://B'
Name
 Language
 Code: 'en-gb'
 Country: 'gb'



2.7 海量数据的交互式分析工具Dr

● 3.数据重组

DocId

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; }}
```


2.7 海量数据的交互式分析工具Dr

● 3.数据重组

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; }}
```

DocId

Links.Backward

2.7 海量数据的交互式分析工具Dr

● 3.数据重组

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward;  
  }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country;  
      optional string Url; }  
  }
```

DocId

Links.Backward

Links.Forward

2.7 海量数据的交互式分析工具Dr

● 3.数据重组

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

DocId

Links.Backward

Links.Forward

Name.Language.Code

2.7 海量数据的交互式分析工具Dr

• 3.数据重组

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

DocId

Links.Backward

Links.Forward

Name.Language.Code

Name.Language.Country

2.7 海量数据的交互式分析工具Dr

• 3.数据重组

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

DocId

Links.Backward

Links.Forward

Name.Language.Code

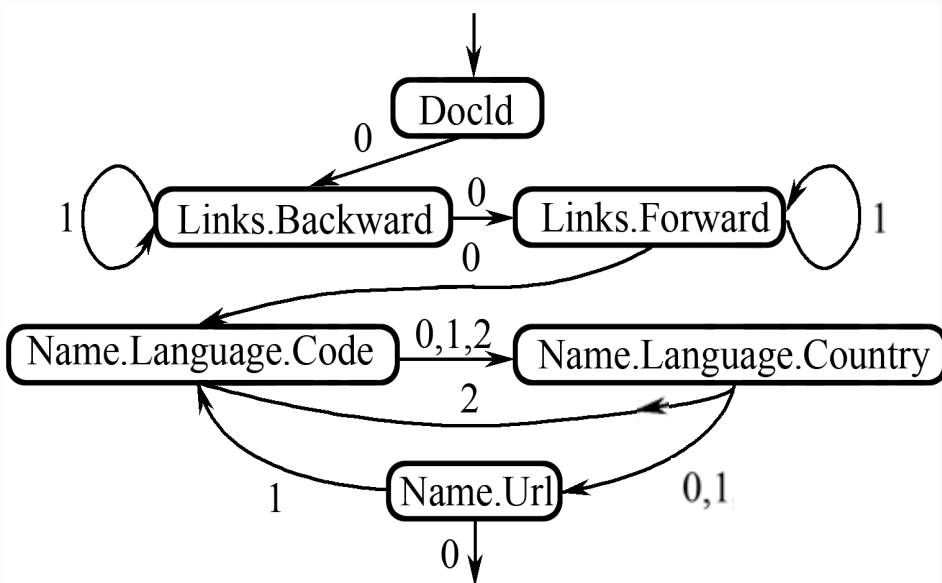
Name.Language.Country

Name.Url

2.7 海量数据的交互式分析工具Dremel

● 3.数据重组

- ◆ 一旦一个reader获取了一个值，就去查看下一个值的重复深度来决定状态如何变化、跳转到哪个reader。
- ◆ 一个FSM状态变化的始终就是一条记录装配的全过程。



DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

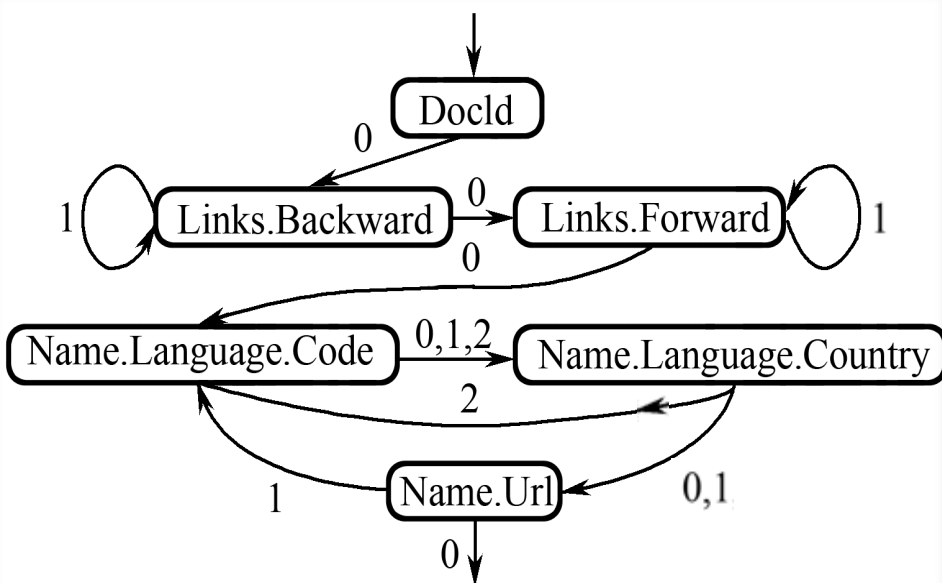
Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

2.7 海量数据的交互式分析工具Dremel

3. 数据重组

- ◆ 思考：如果下一个值的重复深度为0，意味着什么？
- ◆ 意味着当前数值对应的字段是**第一次**出现，没有重复，即，reader是从**其他**字段跳转过来的，不是顺着**当前**字段读取的。
- ◆ 既然不是顺着**当前**字段读取的，那当前字段读完，就必然会跳转到**其他**字段去。



DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

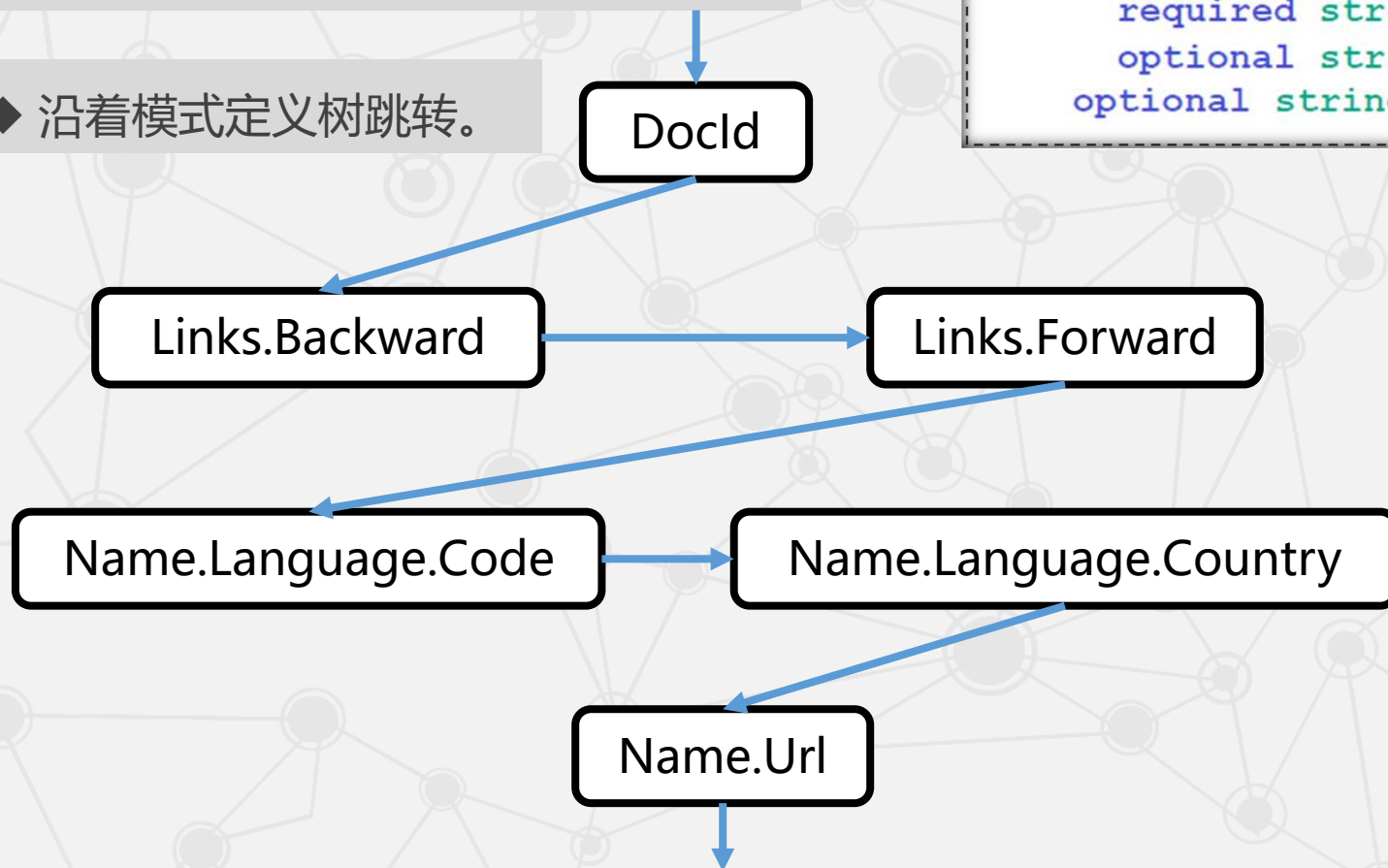
Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

2.7 海量数据的交互式分析工具Dr

● 3.数据重组

◆ 思考：那跳转的**顺序**是什么呢？

◆ 沿着模式定义树跳转。



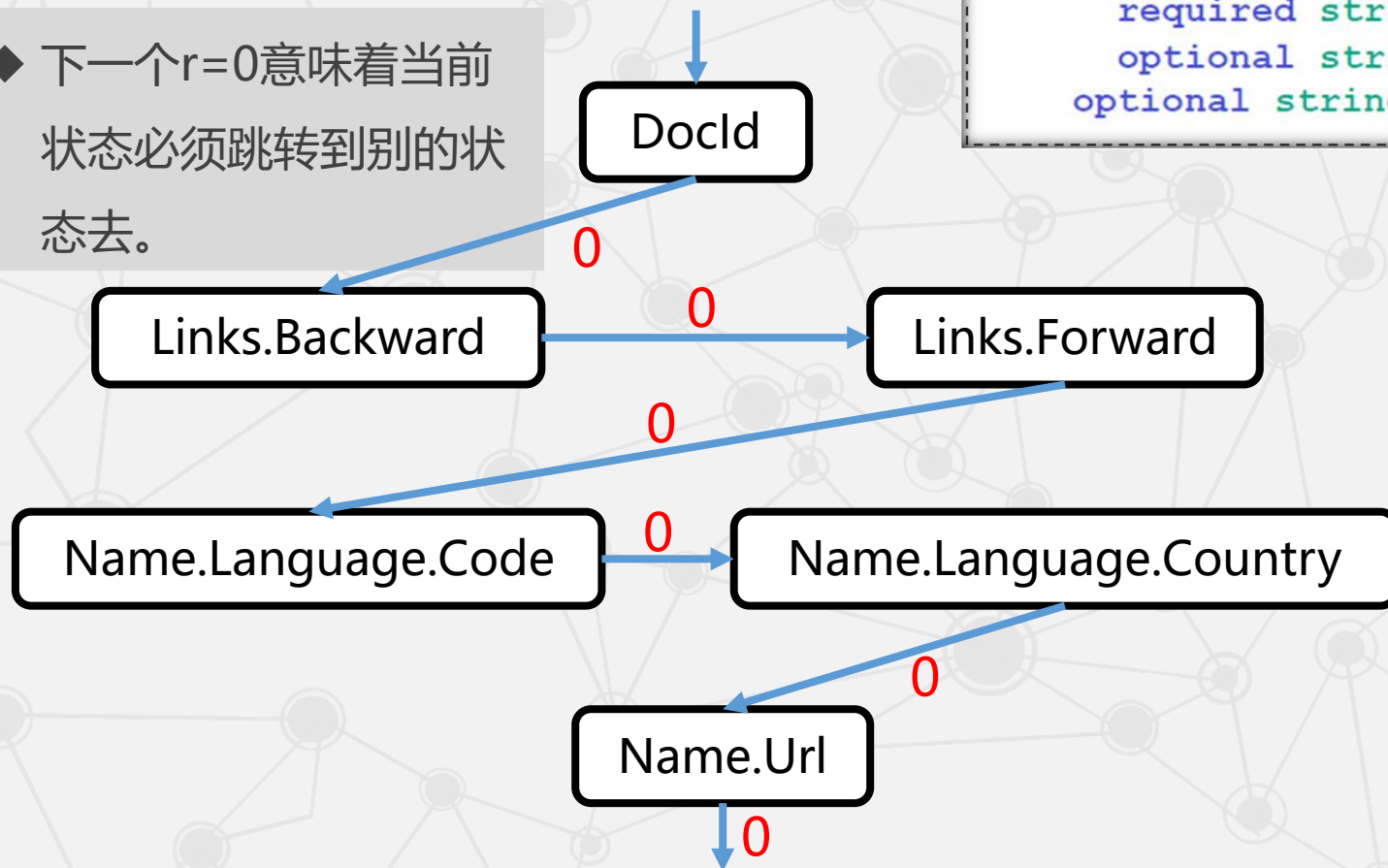
```
message Document {  
  required int64 DocId; 1  
  optional group Links {  
    repeated int64 Backward; 2  
    repeated int64 Forward; } 3  
  repeated group Name {  
    repeated group Language {  
      required string Code; 4  
      optional string Country; 5  
      optional string Url; 6  
    }  
  }  
}
```

2.7 海量数据的交互式分析工具Dr

● 3.数据重组

◆ 思考：那跳转的**条件**又是什么呢？

◆ 下一个 $r=0$ 意味着当前状态必须跳转到别的状态去。



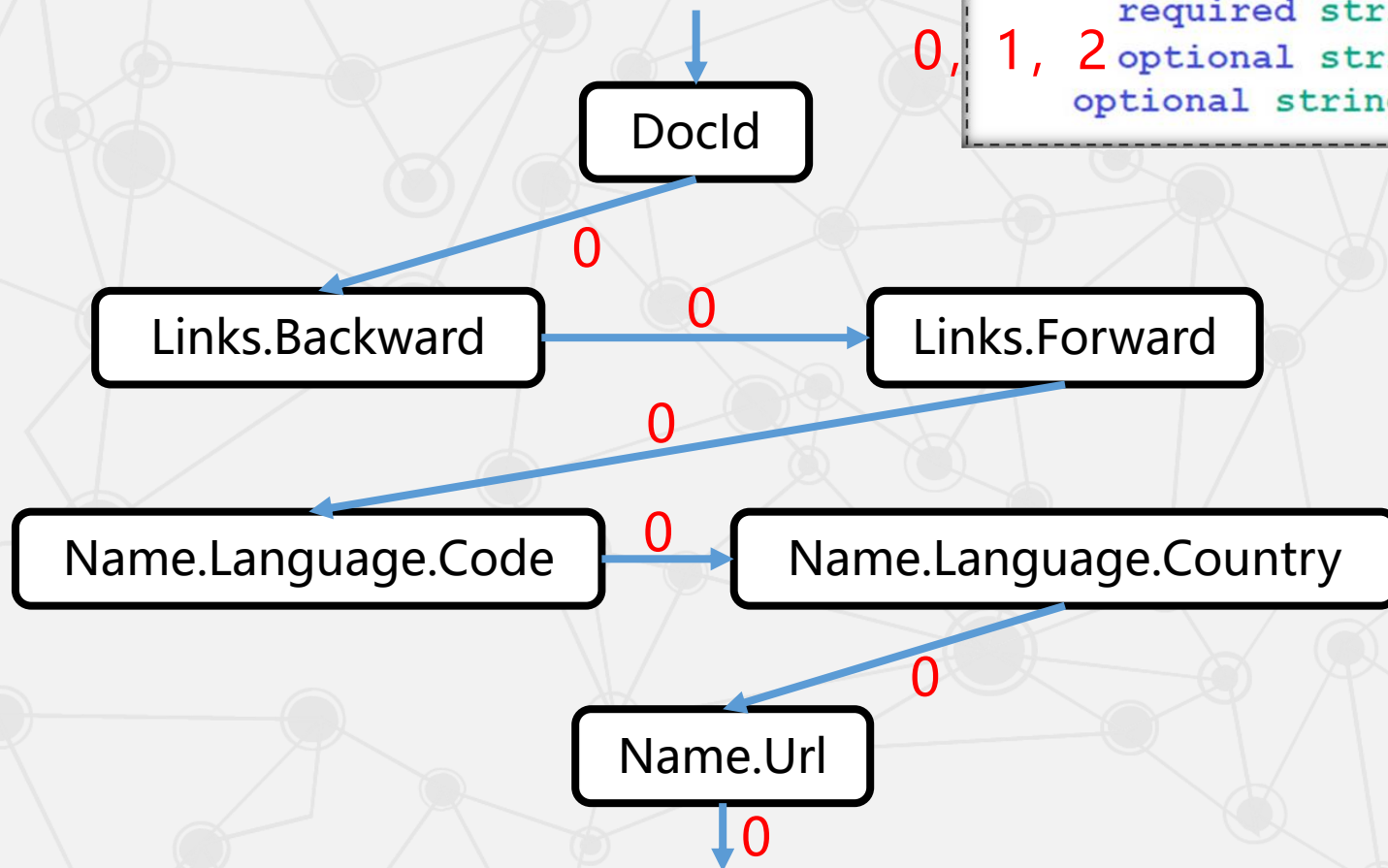
```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } }
```

2.7 海量数据的交互式分析工具Dr

● 3.数据重组

◆ 再来看r的其他可能的取值

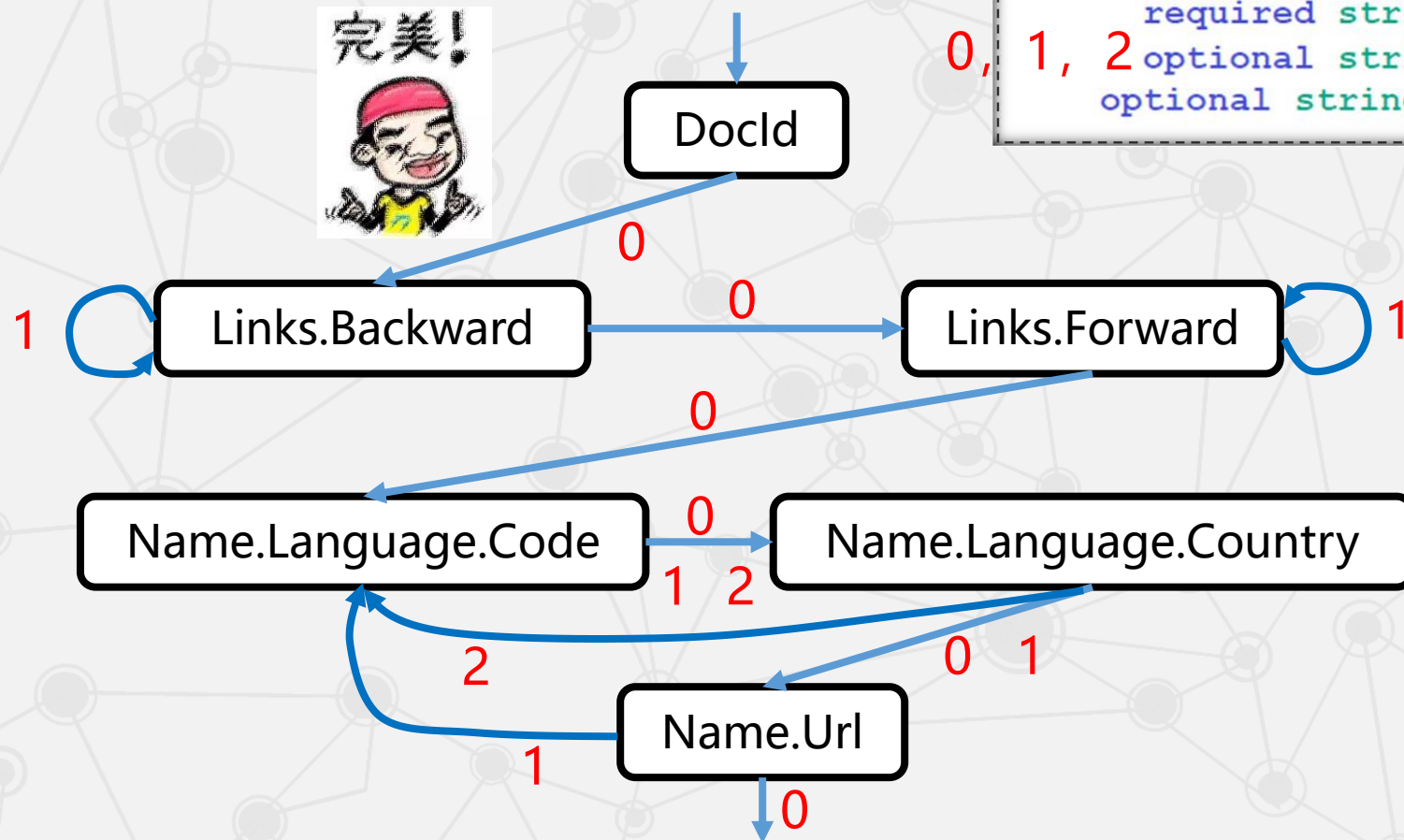
```
message Document {  
  required int64 DocId; 0  
  optional group Links { 0, 1  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name { 0, 1, 2  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; } } 0, 1
```



2.7 海量数据的交互式分析工具Dr

● 3.数据重组

◆ 根据 r ，应如何跳转？



```
message Document {
  required int64 DocId; 0
  optional group Links { 0, 1
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name { 0, 1, 2
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }} 0, 1
```

2.7 海量数据的交互式分析工具Dremel

数据重组: r1的完整数据
重组过程

当前FSM	写入值	下一个重复深度值	动作
DocId (开始)	10	0	跳转至Links.Backward
Links.Backward	NULL	0	跳转至Links.Forward
Links.Forward	20	1	停留在Links.Forward
Links.Forward	40	1	停留在Links.Forward
...	...	...	...

DocId	value	r	d
	10	0	0
	20	0	0

Name.Url	value	r	d
	http://A	0	2
	http://B	1	2
	NULL	1	1
	http://C	0	2

Links.Forward	value	r	d
	20	0	2
	40	1	2
	60	1	2
	80	0	2

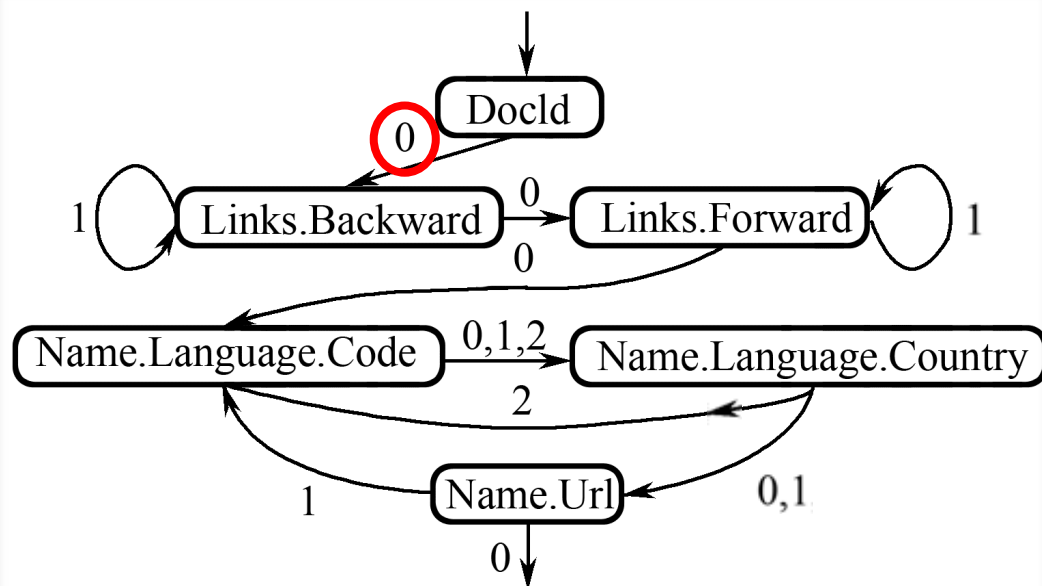
Links.Backward	value	r	d
	NULL	0	1
	10	0	2
	30	1	2

Name.Language.Code	value	r	d
	en-us	0	2
	en	2	2
	NULL	1	1
	en-gb	1	2
	NULL	0	1

Name.Language.Country	value	r	d
	us	0	3
	NULL	2	2
	NULL	1	1
	gb	1	3
	NULL	0	1

Name.Language.Country	gb
Name.Url	NULL

跳转至Name.Language.Code
跳转至Name.Language.Country
跳转至Name.Language.Code
跳转至Name.Language.Country



2.7 海量数据的交互式分析工具Dremel

数据重组: r1的完整数据
重组过程

当前FSM	写入值	下一个重复深度值	动作
DocId (开始)	10	0	跳转至Links.Backward
Links.Backward	NULL	0	跳转至Links.Forward
Links.Forward	20	1	停留在Links.Forward
Links.Forward	40	1	停留在Links.Forward

DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

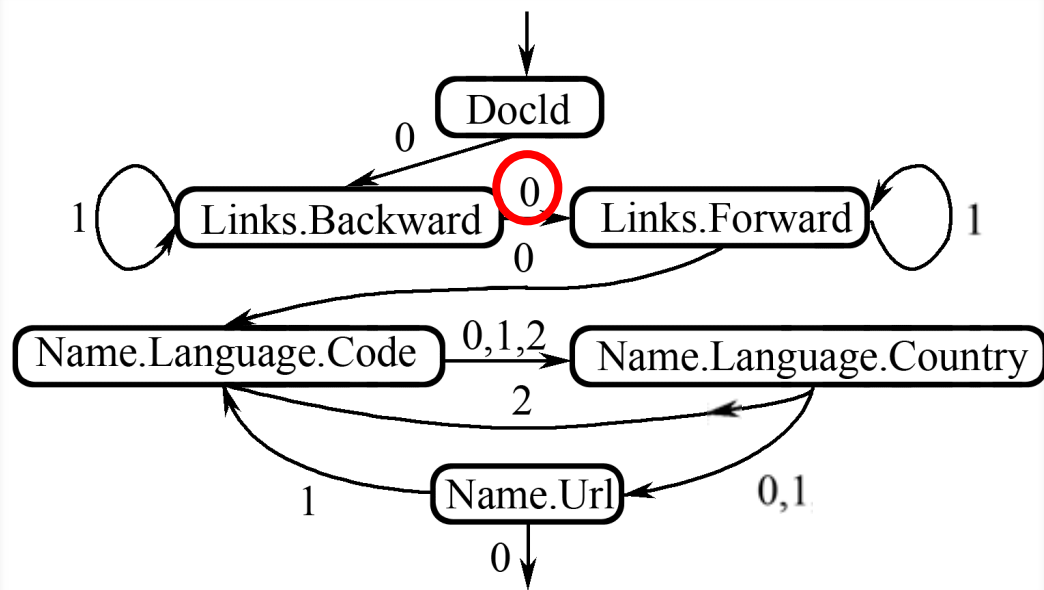
Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

Name.Language.Country	gb
Name.Url	NULL

跳转至Name.Language.Code
跳转至Name.Language.Country
跳转至Name.Language.Code
跳转至Name.Language.Country



2.7 海量数据的交互式分析工具Dremel

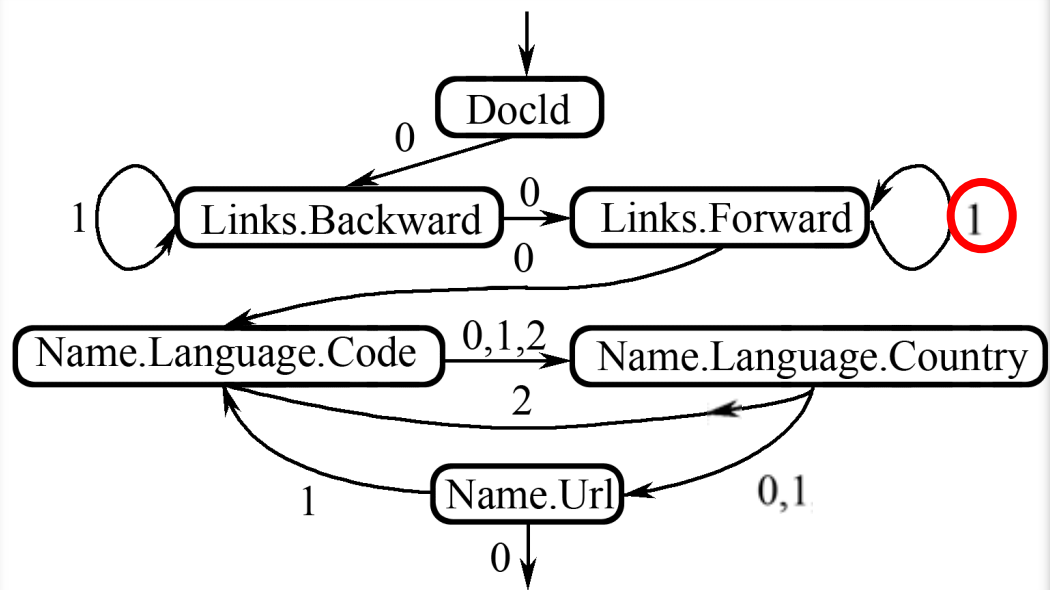
数据重组: r1的完整数据
重组过程

当前FSM	写入值	下一个重复深度值	动作
DocId (开始)	10	0	跳转至Links.Backward
Links.Backward	NULL	0	跳转至Links.Forward
Links.Forward	20	1	停留在Links.Forward
Links.Forward	40	1	停留在Links.Forward
...	...	...	...

DocId	Name.Url	Links.Forward	Links.Backward
value r d	value r d	value r d	value r d
10 0 0	http://A 0 2	20 0 2	NULL 0 1
20 0 0	http://B 1 2	40 1 2	10 0 2
	NULL 1 1	60 1 2	30 1 2
	http://C 0 2	80 0 2	

Name.Language.Code	Name.Language.Country
value r d	value r d
en-us 0 2	us 0 3
en 2 2	NULL 2 2
NULL 1 1	NULL 1 1
en-gb 1 2	gb 1 3
NULL 0 1	NULL 0 1

Name.Language.Country	gb
Name.Url	NULL



2.7 海量数据的交互式分析工具Dremel

数据重组: r1的完整数据
重组过程

当前FSM	写入值	下一个重复深度值	动作
DocId (开始)	10	0	跳转至Links.Backward
Links.Backward	NULL	0	跳转至Links.Forward
Links.Forward	20	1	停留在Links.Forward
Links.Forward	40	1	停留在Links.Forward
...	...	...	...
...	...	...	跳转至Name.Language.Code
...	...	...	跳转至Name.Language.Country
...	...	...	跳转至Name.Language.Code
...	...	...	跳转至Name.Language.Country

DocId		
value	r	d
10	0	0
20	0	0

Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

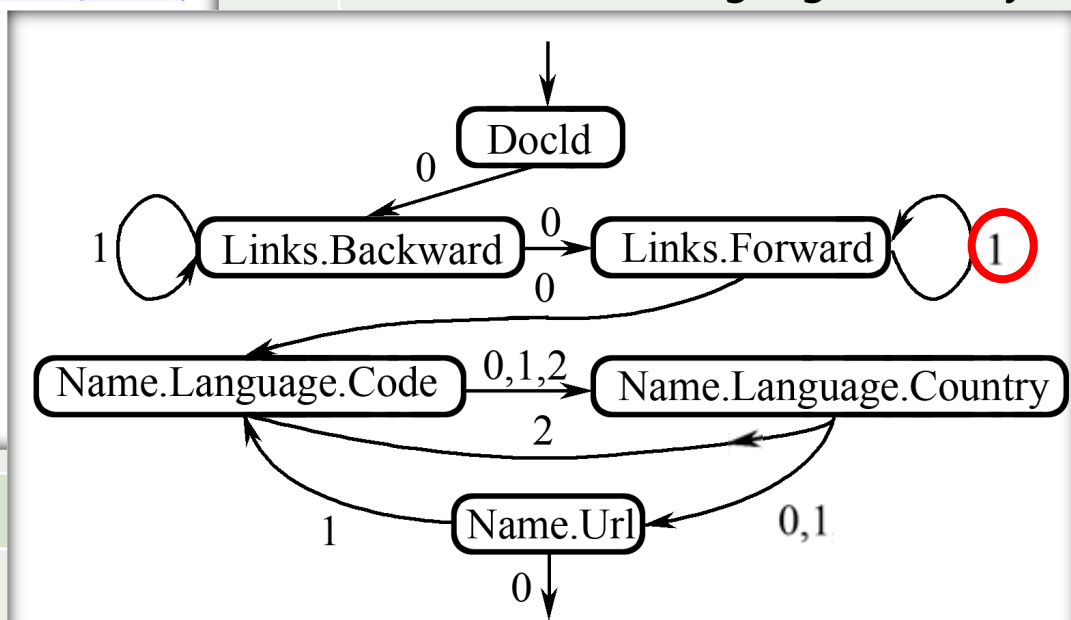
Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

Name.Language.Country	gb
Name.Url	NULL



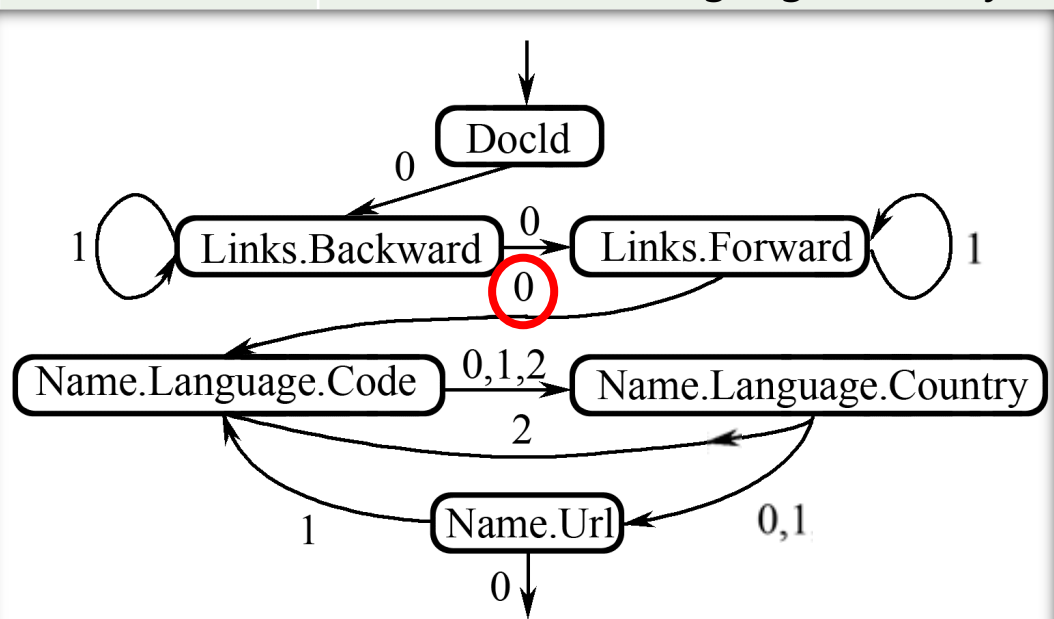
2.7 海量数据的交互式分析工具Dremel

数据重组: r1的完整数据
重组过程

DocId			Name.Url			Links.Forward			Links.Backward		
value	r	d	value	r	d	value	r	d	value	r	d
10	0	0	http://A	0	2	20	0	2	NULL	0	1
20	0	0	http://B	1	2	40	1	2	10	0	2
			NULL	1	1	60	1	2	30	1	2
			http://C	0	2	80	0	2			

深度值	动作
	跳转至Links.Backward
	跳转至Links.Forward
	停留在Links.Forward
	停留在Links.Forward
	跳转至Name.Language.Code
	跳转至Name.Language.Country
	跳转至Name.Language.Code
	跳转至Name.Language.Country

Links.Forward	40	1
Links.Forward	60	0
Name.Language.Code	en-us	2
Name.Language.Country	us	2
Name.Language.Code	en	1
Name.Language.Country	NULL	
Name.Url	http://A	
Name.Language.Code	NULL	
Name.Language.Country	NULL	
Name.Url	http://B	
Name.Language.Code	en-gb	
Name.Language.Country	gb	
Name.Url	NULL	



Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

深度值

动作

跳转至Links.Backward

跳转至Links.Forward

停留在Links.Forward

停留在Links.Forward

跳转至Name.Language.Code

跳转至Name.Language.Country

跳转至Name.Language.Code

跳转至Name.Language.Country

Links.Forward

60

0

Name.Language.Code

en-us

2

Name.Language.Country

us

2

Name.Language.Code

en

1

Name.Language.Country

NULL

Name.Url

http://A

Name.Language.Code

NULL

Name.Language.Country

NULL

Name.Url

http://B

Name.Language.Code

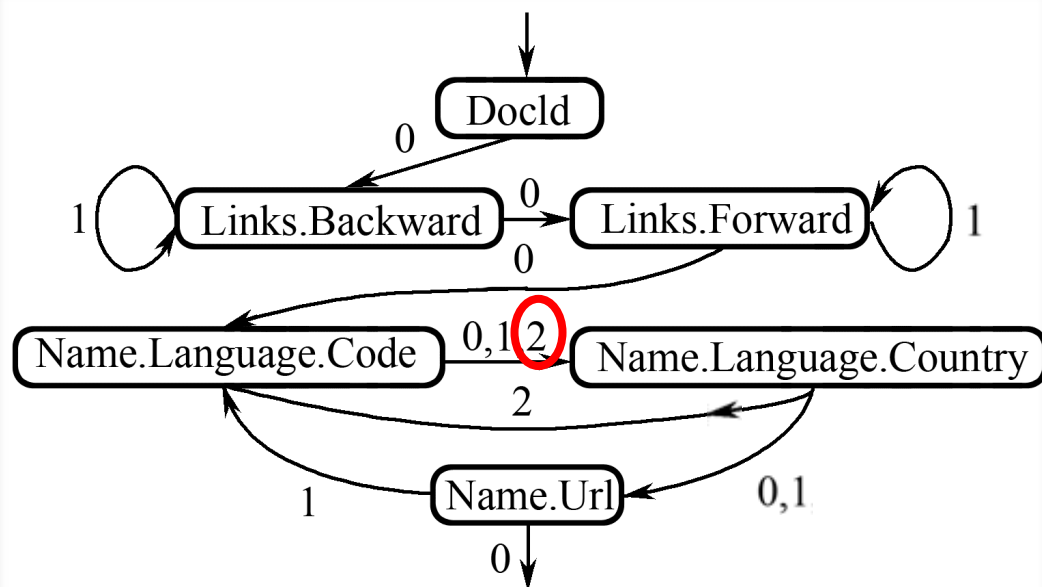
en-gb

Name.Language.Country

gb

Name.Url

NULL



Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

深度值

动作

跳转至Links.Backward

跳转至Links.Forward

停留在Links.Forward

停留在Links.Forward

跳转至Name.Language.Code

跳转至Name.Language.Country

跳转至Name.Language.Code

跳转至Name.Language.Country

Links.Forward

60

0

Name.Language.Code

en-us

2

Name.Language.Country

us

2

Name.Language.Code

en

1

Name.Language.Country

NULL

Name.Url

http://A

Name.Language.Code

NULL

Name.Language.Country

NULL

Name.Url

http://B

Name.Language.Code

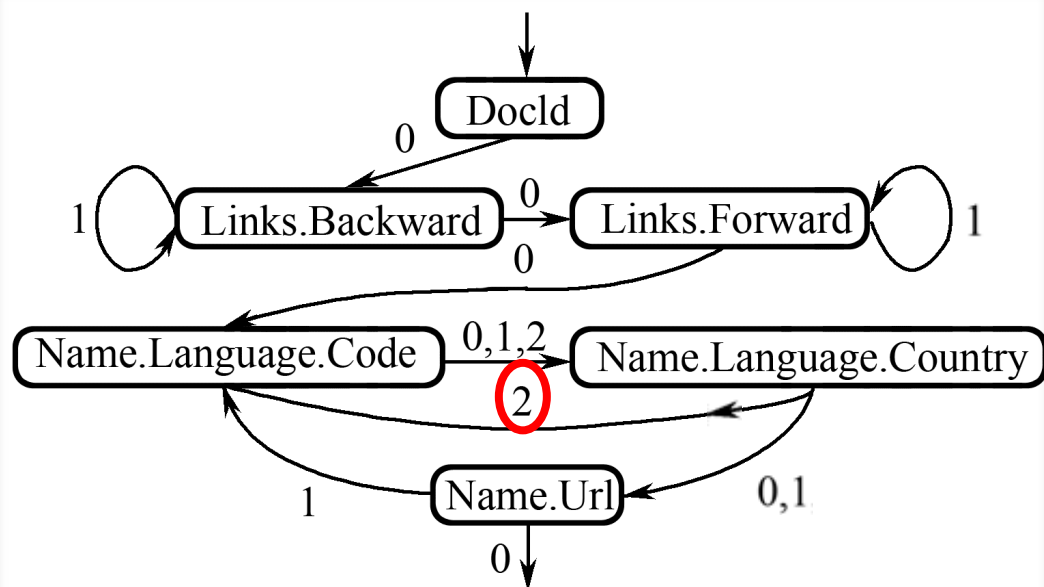
en-gb

Name.Language.Country

gb

Name.Url

NULL



Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

深度值

动作

跳转至Links.Backward

跳转至Links.Forward

停留在Links.Forward

停留在Links.Forward

跳转至Name.Language.Code

跳转至Name.Language.Country

跳转至Name.Language.Code

跳转至Name.Language.Country

Links.Forward

60

0

Name.Language.Code

en-us

2

Name.Language.Country

us

2

Name.Language.Code

en

1

Name.Language.Country

NULL

Name.Url

http://A

Name.Language.Code

NULL

Name.Language.Country

NULL

Name.Url

http://B

Name.Language.Code

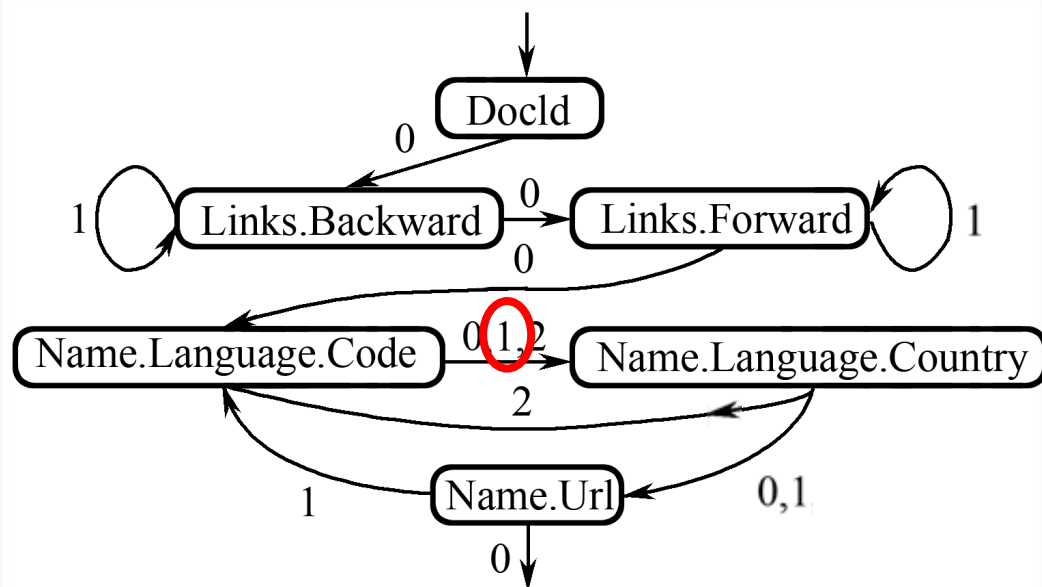
en-gb

Name.Language.Country

gb

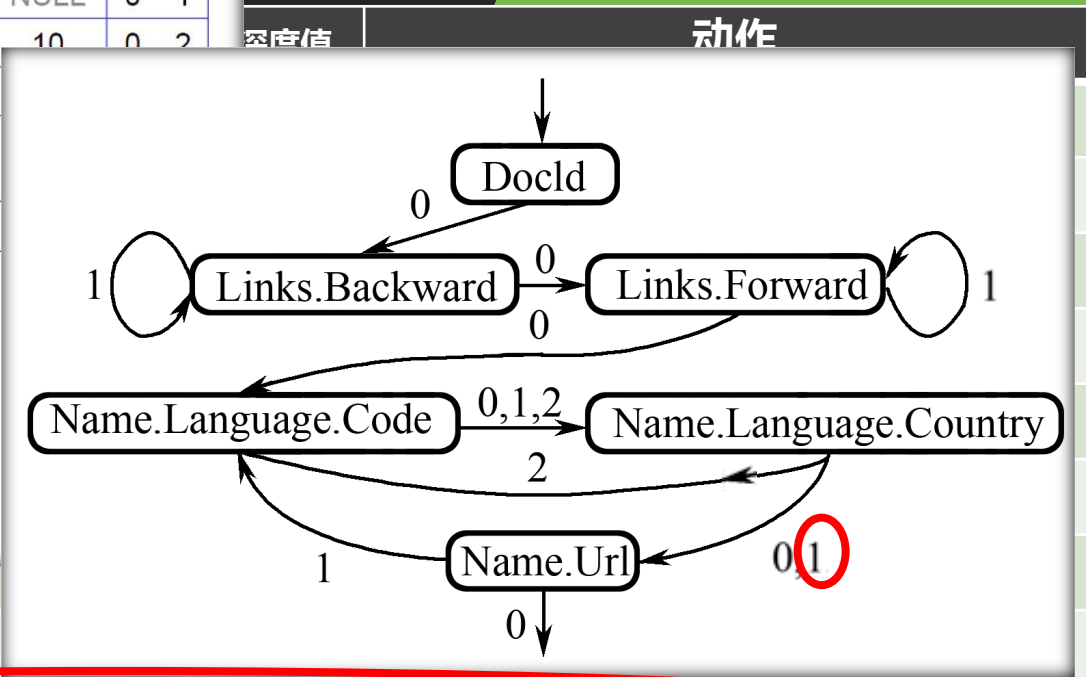
Name.Url

NULL



DocId	Name.Url			Links.Forward			Links.Backward				
value	r	d	value	r	d	value	r	d	value	r	d
10	0	0	http://A	0	2	20	0	2	NULL	0	1
20	0	0	http://B	1	2	40	1	2	10	0	2
			NULL	1	1	60	1	2			
			http://C	0	2	80	0	2			

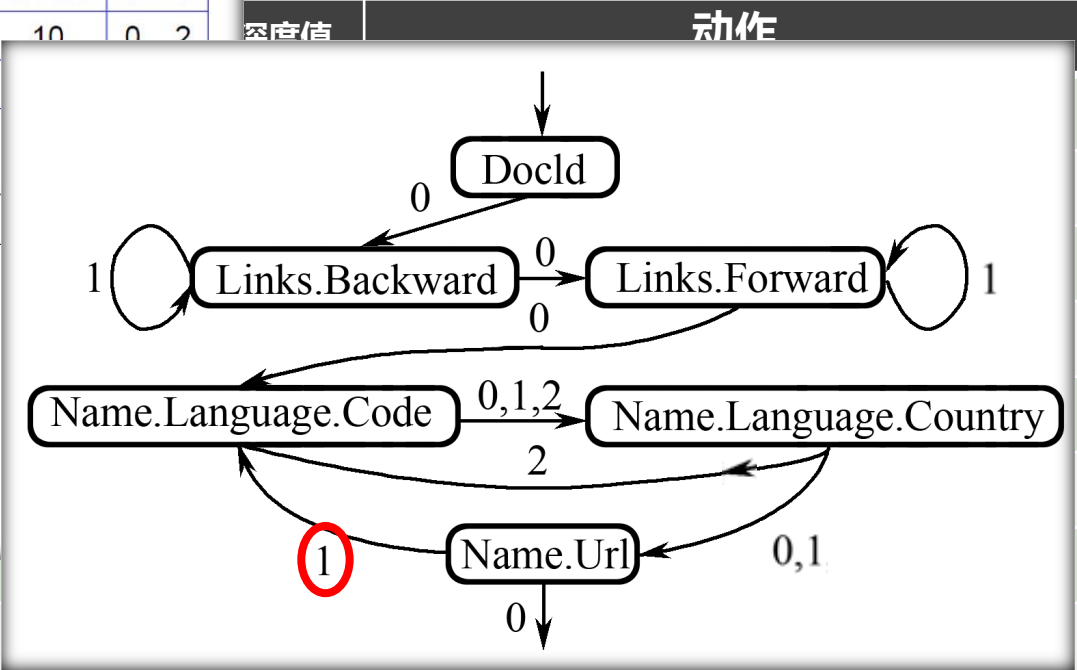
Name.Language.Code			Name.Language.Country		
value	r	d	value	r	d
en-us	0	2	us	0	3
en	2	2	NULL	2	2
NULL	1	1	NULL	1	1
en-gb	1	2	gb	1	3
NULL	0	1	NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId			Name.Url			Links.Forward			Links.Backward		
value	r	d	value	r	d	value	r	d	value	r	d
10	0	0	http://A	0	2	20	0	2	NULL	0	1
20	0	0	http://B	1	2	40	1	2	10	0	2
			NULL	1	1	60	1	2			
			http://C	0	2	80	0	2			

Name.Language.Code			Name.Language.Country		
value	r	d	value	r	d
en-us	0	2	us	0	3
en	2	2	NULL	2	2
NULL	1	1	NULL	1	1
en-gb	1	2	gb	1	3
NULL	0	1	NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId		
value	r	d
10	0	0
20	0	0

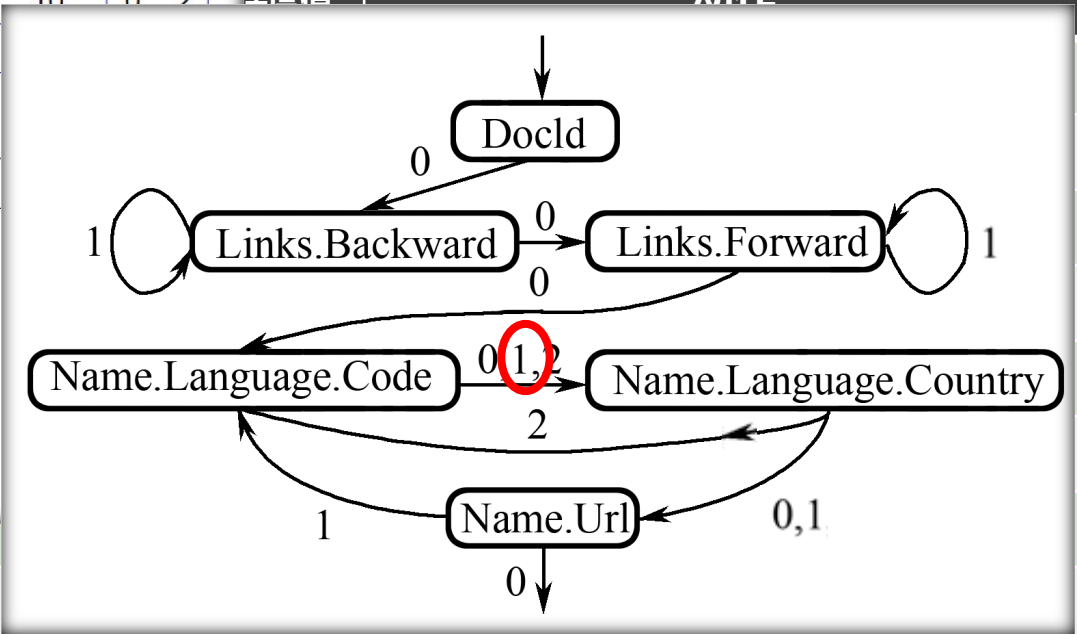
Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId		
value	r	d
10	0	0
20	0	0

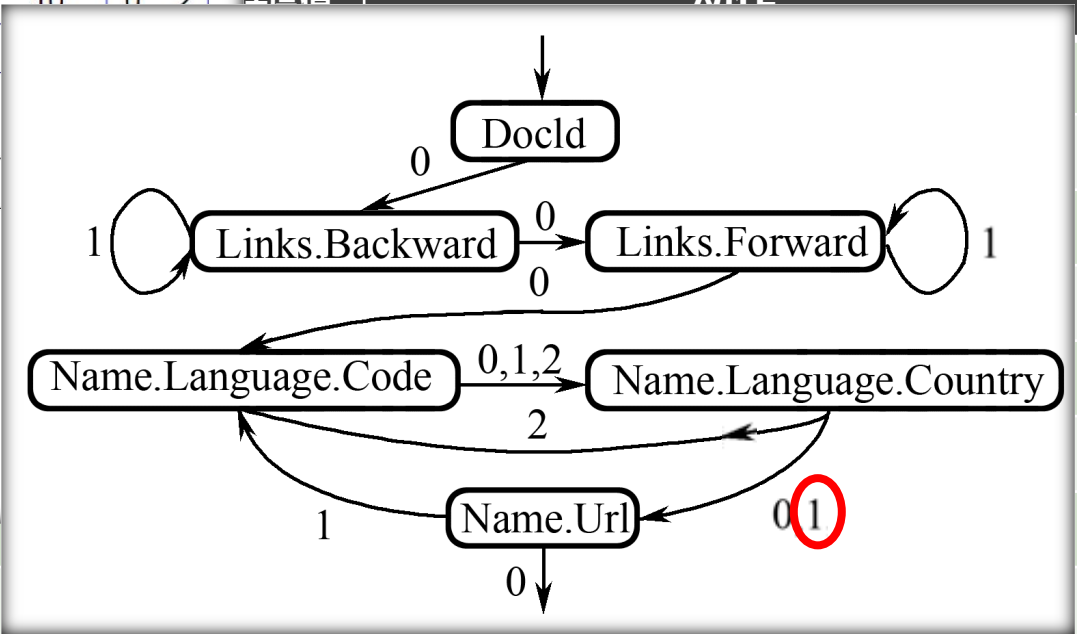
Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId		
value	r	d
10	0	0
20	0	0

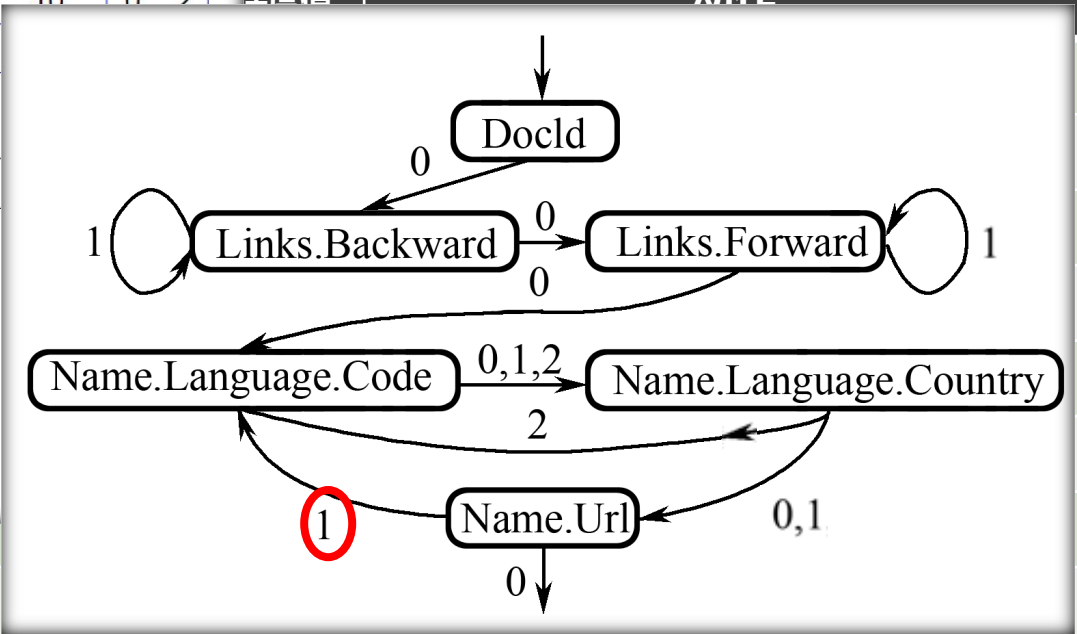
Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId		
value	r	d
10	0	0
20	0	0

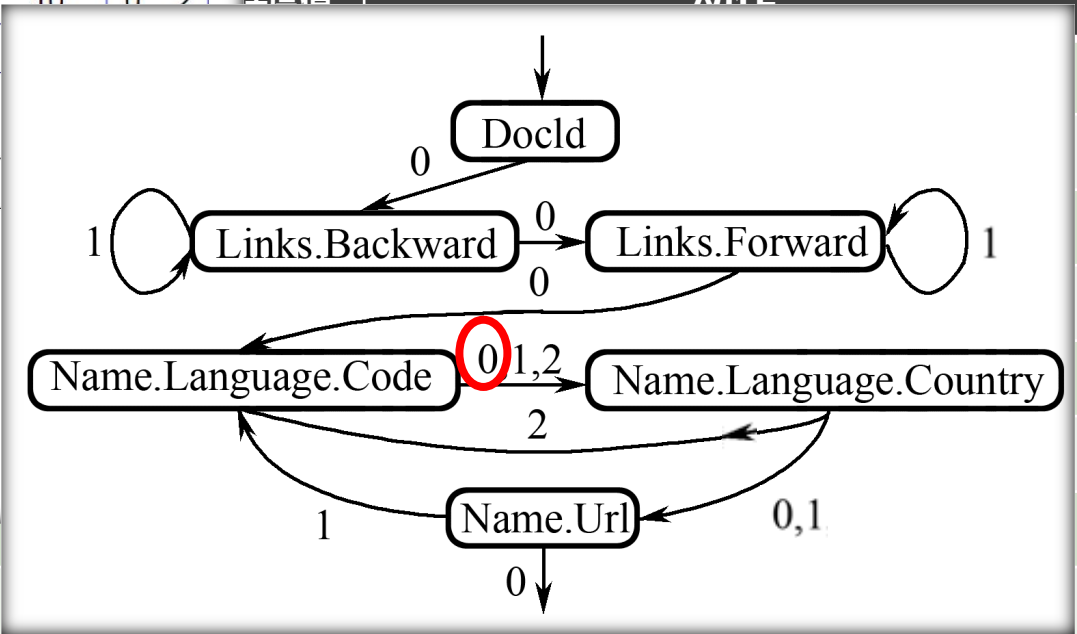
Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



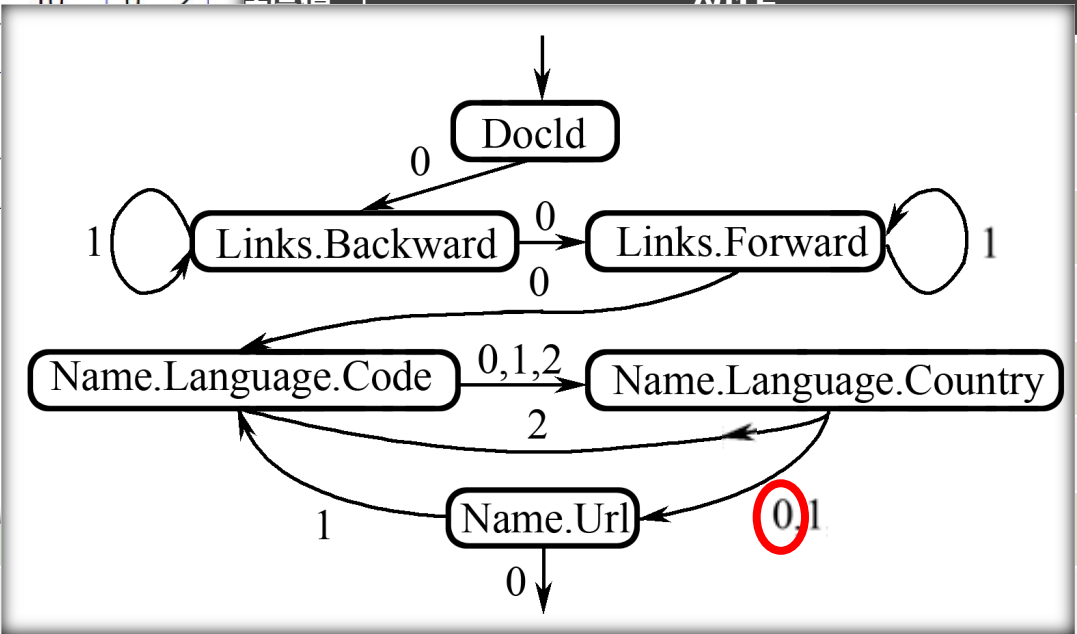
Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId			Name.Url			Links.Forward			Links.Backward		
value	r	d	value	r	d	value	r	d	value	r	d
10	0	0	http://A	0	2	20	0	2	NULL	0	1
20	0	0	http://B	1	2	40	1	2	10	0	2
			NULL	1	1	60	1	2			
			http://C	0	2	80	0	2			

Name.Language.Code			Name.Language.Country		
value	r	d	value	r	d
en-us	0	2	us	0	3
en	2	2	NULL	2	2
NULL	1	1	NULL	1	1
en-gb	1	2	gb	1	3
NULL	0	1	NULL	0	1

Dremel
数据重组: r1的完整数据重组过程

密度值
动作



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

DocId		
value	r	d
10	0	0
20	0	0

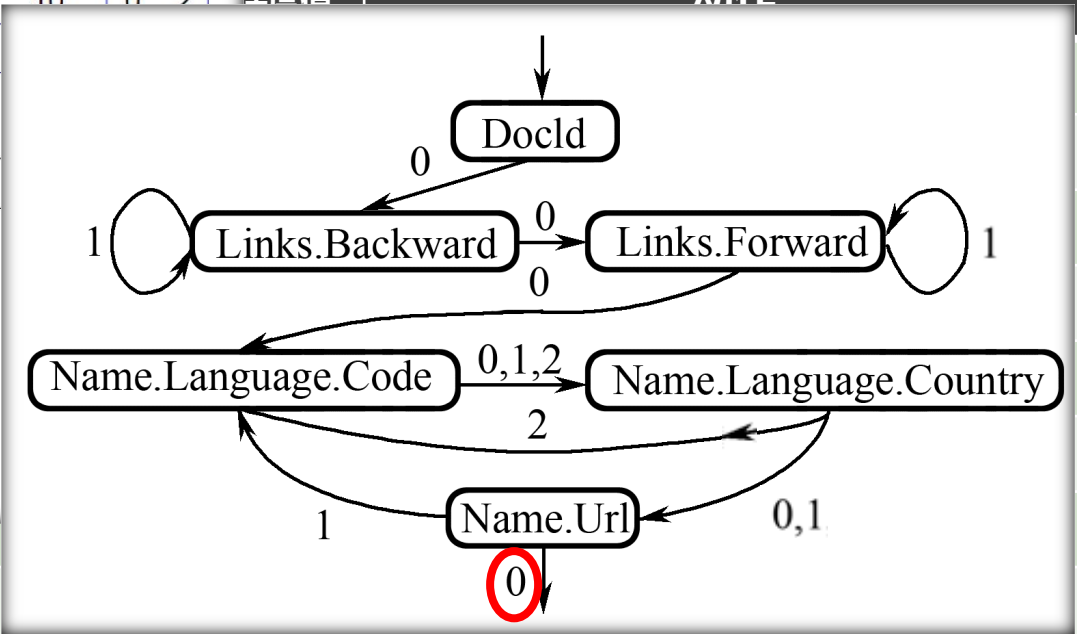
Name.Url		
value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward		
value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward		
value	r	d
NULL	0	1
10	0	2

Name.Language.Code		
value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



Name.Language.Country	us		
Name.Language.Code	en		
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

当前FSM	写入值
DocId (开始)	10
Links.Backward	NULL
Links.Forward	20
Links.Forward	40
Links.Forward	60
Name.Language.Code	en-us
Name.Language.Country	us
Name.Language.Code	en
Name.Language.Country	NULL
Name.Url	http://A
Name.Language.Code	NULL
Name.Language.Country	NULL
Name.Url	http://B
Name.Language.Code	en-gb
Name.Language.Country	gb
Name.Url	NULL

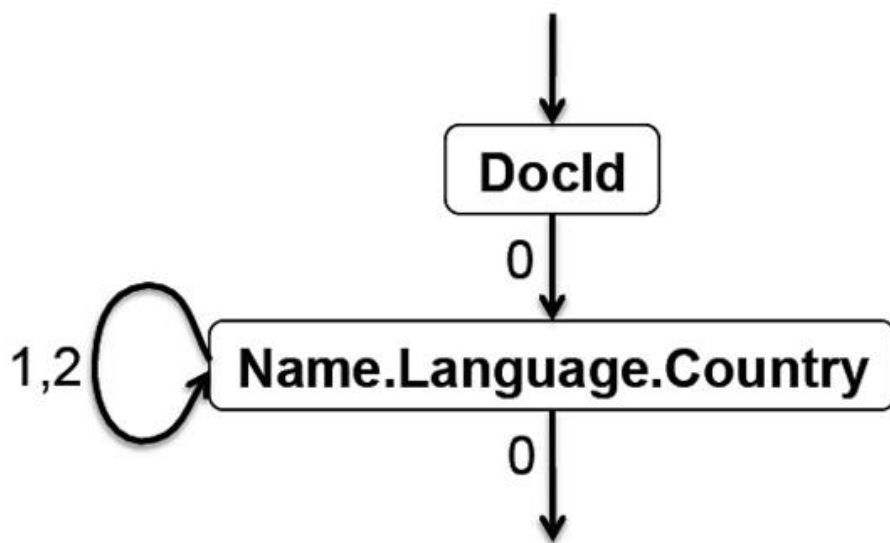
DocId: 10	r ₁
Links	
Forward: 20	
Forward: 40	
Forward: 60	
Name	
Language	
Code: 'en-us'	
Country: 'us'	
Language	
Code: 'en'	
Url: 'http://A'	
Name	
Url: 'http://B'	
Name	
Language	
Code: 'en-gb'	
Country: 'gb'	

动作
ks.Backward
ks.Forward
ks.Forward
ks.Forward
Language.Code
anguage.Country
Language.Code
anguage.Country
Name.Url
Language.Code
anguage.Country
Name.Url
Language.Code
anguage.Country
Name.Url
結束

2.7 海量数据的交互式分析工具Dremel

• 3. 数据重组

如果具体的查询中不是涉及所有列，而是仅涉及很少的列的话，上述数据重组的过程会更加便利，下图中仅仅涉及DocId和Name.Language.Country的有限状态机。



DocId: 10 S_1

Name

Language

Country: 'us'

Language

Name

Language

Country: 'gb'

DocId: 20 S_2

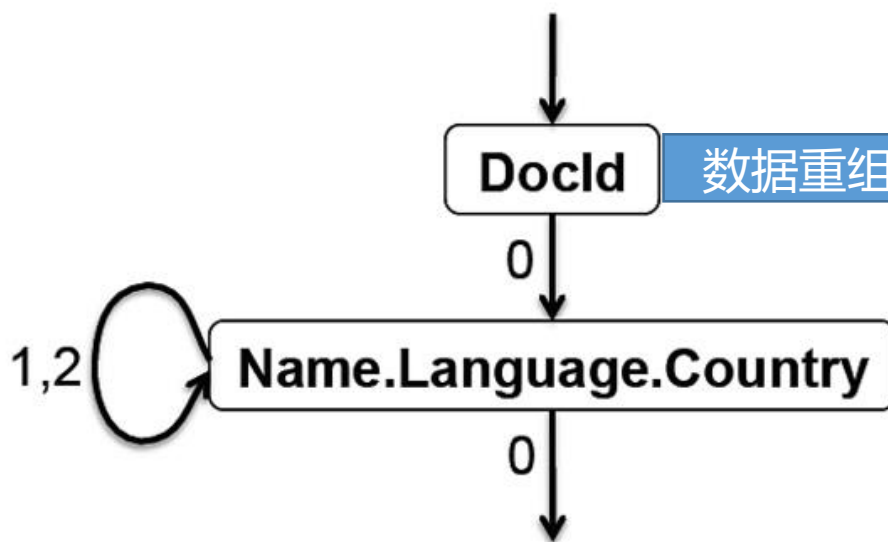
Name

2.7 海量数据的交互式分析工具Dremel

• 3. 数据重组

DocId		
value	r	d
10	0	0
20	0	0

Name.Language.Country		
value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1



DocId: 10	S ₁
Name	
Language	
Country: 'us'	
Language	
Name	
Language	
Country: 'gb'	

DocId: 20	S ₂
Name	

数据的无损表示

2.7 海量数据的交互式分析工具Dremel

● 3. 数据重组

核心的思想如下：

设置 t 为当前字段读取器的当前值 f 所返回的下一个重复深度。

在模式树中，找到它在深度 t 的祖先，然后选择该祖先节点的第一个叶子字段 n 。

由此得到一个FSM状态变化 $(f,t) \rightarrow n$ 。

有限状态机的构造算法

```
1 Record AssembleRecord(FieldReaders[] readers):
2   record = create a new record
3   lastReader = select the root field reader in readers
4   reader = readers[0]
5   while reader has data
6     Fetch next value from reader
7     if current value is not NULL
8       MoveToLevel(tree level of reader, reader)
9       Append reader's value to record
10    else
11      MoveToLevel(full definition level of reader, reader)
12    end if
13    reader = reader that FSM transitions to
14      when reading next repetition level from reader
15    ReturnToLevel(tree level of reader)
16  end while
17  ReturnToLevel(0)
18  End all nested records
19  return record
20 end procedure
21
22 MoveToLevel(int newLevel, FieldReader nextReader):
23   End nested records up to the level of the lowest common ancestor
24   of lastReader and nextReader.
25   Start nested records from the level of the lowest common ancestor
26   up to newLevel.
27   Set lastReader to the one at newLevel.
28 end procedure
29
30 ReturnToLevel(int newLevel) {
31   End nested records up to newLevel.
32   Set lastReader to the one at newLevel.
33 end procedure
```


2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

► 2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

Dremel的类SQL查询输入的是一个或多个嵌套结构的表以及相应的模式，而输出的结果是一个嵌套结构的表以及相应的模式。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

Id: 10
Name
 Cnt: 2
 Language
 Str: 'http://A,en-us'
 Str: 'http://A,en'
Name
 Cnt: 0

t_1

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }  
  }  
}
```

图中显示了：

查询语言

查询结果模式

最终输出的
数据模式

图2-47 Dremel的SQL查询语句实例

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

下图中的查询执行在图2-41中的 $t = \{r_1, r_2\}$ 上，实际执行了**投影**（SELECT）、**选择**（WHERE）和**记录内聚合**（COUNT）等操作。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

Id: 10	t_1
Name	
Cnt: 2	
Language	
Str: 'http://A,en-us'	
Str: 'http://A,en'	
Name	
Cnt: 0	

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }}};
```

图2-47 Dremel的SQL查询语句实例

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

选择 (**WHERE**) 操作就是对不满足指定条件的分支进行剪枝。例如，只有当 Name.Url 非空且满足正则表达式 “^http” 才被保留。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

Id: 10 **t₁**
Name
 Cnt: 2
 Language
 Str: 'http://A,en-us'
 Str: 'http://A,en'
Name
 Cnt: 0

DocId: 10 **r₁**
Links
 Forward: 20
 Forward: 40
 Forward: 60
Name
 Language
 Code: 'en-us'
 Country: 'us'
 Language
 Code: 'en'
Url: 'http://A'
Name
Url: 'http://B'
Name
 Language
 Code: 'en-gb'
 Country: 'gb'

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country;  
      optional string Url; } }  
}
```

DocId: 20 **r₂**
Links
 Backward: 10
 Backward: 30
 Forward: 80
Name
Url: 'http://C'

图2-47 Dremel的SQL

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

投影 (**SELECT**) 子句中的每个标量表达式都会投影为一个值，此值的嵌套深度和表达式中重复字段最多的保持一致。所以，Str值的嵌套深度与Name.Language.Code相同。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

r=1 **r=2**

Id: 10

Name

Cnt: 2

t₁

Na

```
message Document {  
  required int64 DocId;  
  optional group Links {  
    repeated int64 Backward;  
    repeated int64 Forward; }  
  repeated group Name {  
    repeated group Language {  
      required string Code;  
      optional string Country; }  
    optional string Url; }  
}
```

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }  
    }  
}
```

SQL查询语句实例

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

记录内聚合 (COUNT) 是指每个Name子记录都会执行此聚合。将 Name.Language.Code出现的COUNT投影为每个Name下的Cnt值。

```
SELECT DocId AS Id,  
       COUNT (Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP (Name.Url, '^http') AND DocId < 20;
```

Id: 10	t_1
Name	
<u>Cnt: 2</u>	
Language	
Str: 'http://A,en-us'	
Str: 'http://A,en'	
Name	
<u>Cnt: 0</u>	

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }}};
```

图2-47 Dremel的SQL查询语句实例

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

记录内聚合 (COUNT) 是指每个Name子记录都会执行此聚合。将 Name.Language.Code出现的COUNT投影为每个Name下的Cnt值。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

Id: 10 **t₁**
Name
 Cnt: 2
 Language
 Str: 'http://A,en-us'
 Str: 'http://A,en'
Name
 Cnt: 0

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str;  
    }  
  }  
}
```

DocId: 10 **r₁**
Links
 Forward: 20
 Forward: 40
 Forward: 60
Name
 Language
 Code: 'en-us'
 Country: 'us'
 Language
 Code: 'en'
 Url: 'http://A'
Name
 Url: 'http://B'
Name
 Language
 Code: 'en-gb'
 Country: 'gb'

图2-47 Dremel的SQL查询语句实例

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

Dremel利用**多层次服务树** (multi-level service tree) 的概念来执行查询操作

根服务器

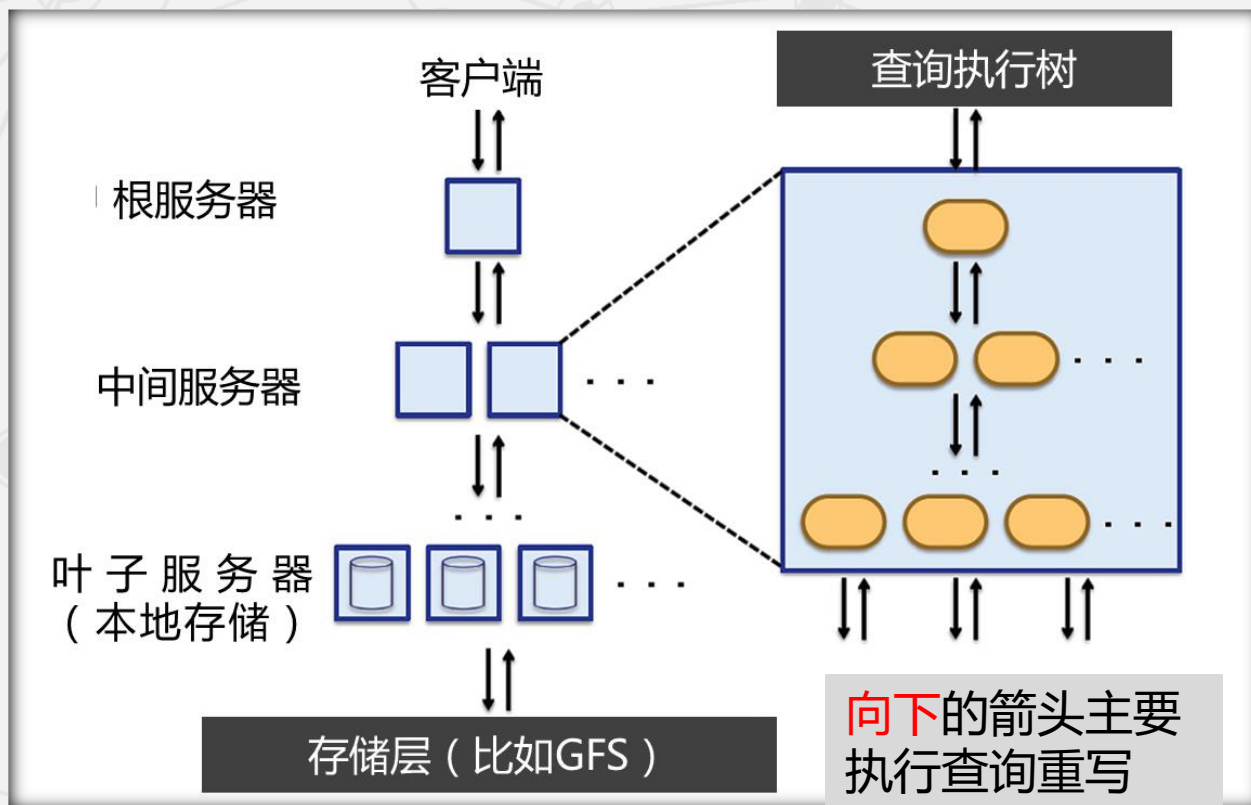
接受客户端发出的请求，读取相应的**元数据**，将请求转发至中间服务器。

中间服务器

负责查询**中间结果**的聚集。

叶子服务器

负责数据来源。既可以读取**本地数据**，也可以向统一的**存储层**发出数据请求来获取数据。



向下的箭头主要
执行查询重写

向上的箭头主要是
查询执行和聚集

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

满足条件的记录表r1

选择条件

大多数的Dremel查询都是简单的聚集操作（不涉及join等复杂操作），
例如：SELECT A, COUNT(B) FROM T GROUP BY A

```
SELECT DocId AS Id,  
      COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
      Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

```
Id: 10  
Name  
  Cnt: 2  
  Language  
    Str: 'http://A,en-us'  
    Str: 'http://A,en'  
Name  
  Cnt: 0
```

t₁

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }}}}
```

2.7 海量数据的交互式分析工具Dremel

2.7.4 查询语言与执行

式 (2-2) : $\text{SELECT } A, \text{COUNT}(B) \text{ FROM } T \text{ GROUP BY } A$

系统沿着多层级服务树从上到下不断地执行查询重写：

$\text{SELECT } A, \text{COUNT}(c) \text{ FROM } (R_1^1 \text{ UNION ALL } \dots R_n^1) \text{ GROUP BY } A$

其中, R_i^1 是树中第1层的节点1~n返回的子查询结果 (根节点是第0层)。 R_i^1 的查询表达式为:

$R_i^1 = \text{SELECT } A, \text{COUNT}(B) \text{ AS } c \text{ FROM } T_i^1 \text{ GROUP BY } A$

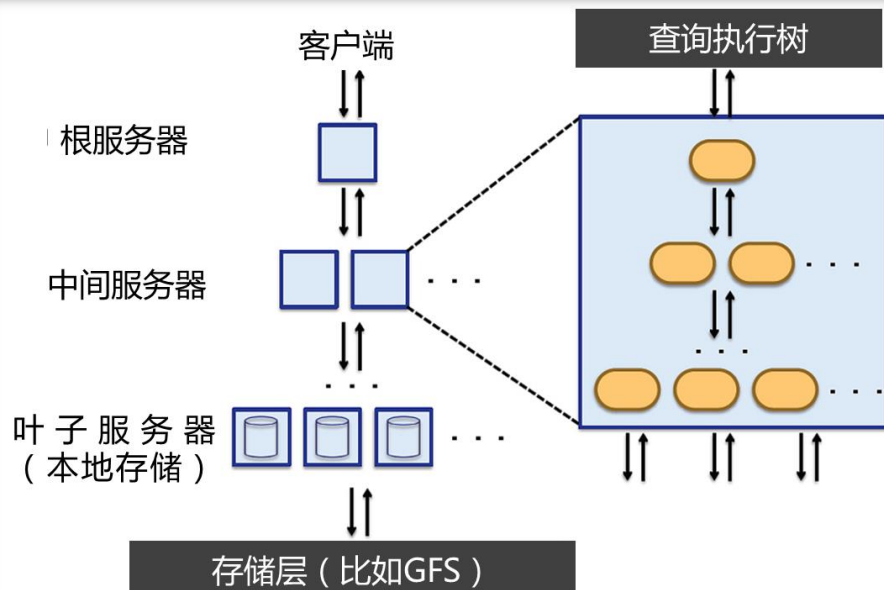
R_1

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

R_2

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```



2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

其中, R_1^1 是树中第1层的节点1~n返回的子查询结果 (根节点是第0层)。 R_i^1 的查询表达式为:

$$R_i^1 = \text{SELECT } A, \text{COUNT}(B) \text{ AS } c \text{ FROM } T_i^1 \text{ GROUP BY } A$$

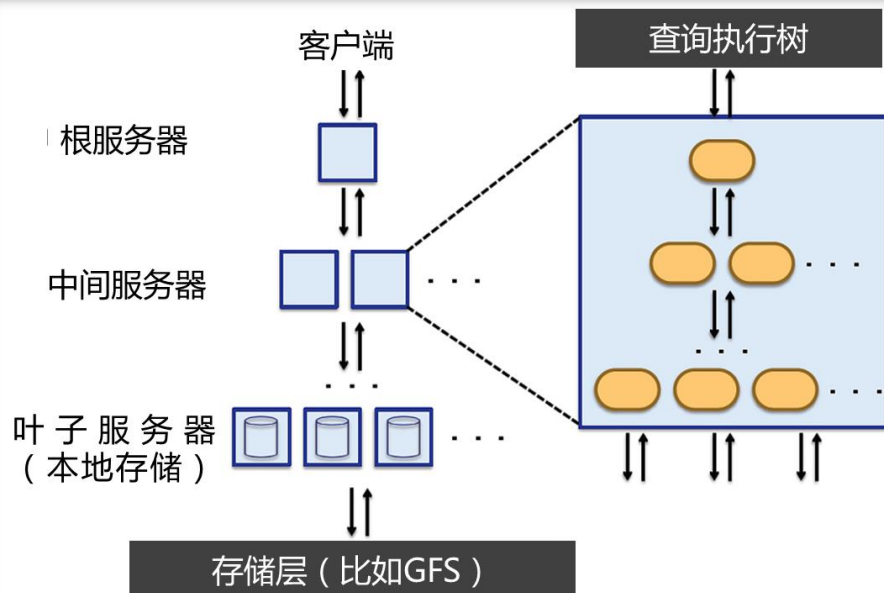
R_1^1

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

R_2^1

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```



2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

其中, R_1^1 是树中第1层的节点1~n返回的子查询结果 (根节点是第0层)。 R_i^1 的查询表达式为:

$$R_i^1 = \text{SELECT } A, \text{COUNT}(B) \text{ AS } c \text{ FROM } T_i^1 \text{ GROUP BY } A$$

Dremel中的数据是分布式存储的, 因此每一层查询涉及的数据都是被水平划分后, 存储在多个服务器上。 T_i^1 表示T的第i个分区的数据。

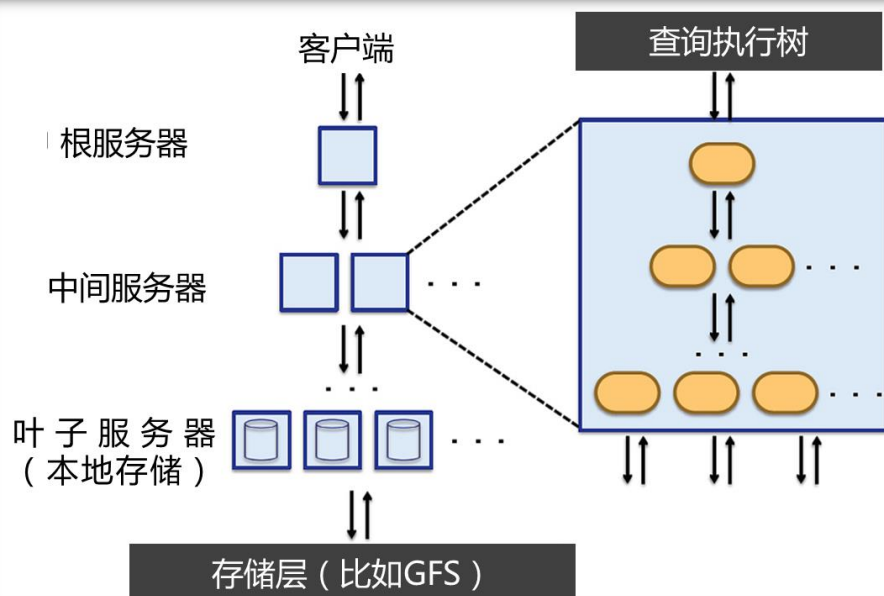
r_1

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

r_2

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```



2.7 海量数据的交互式分析工具Dremel

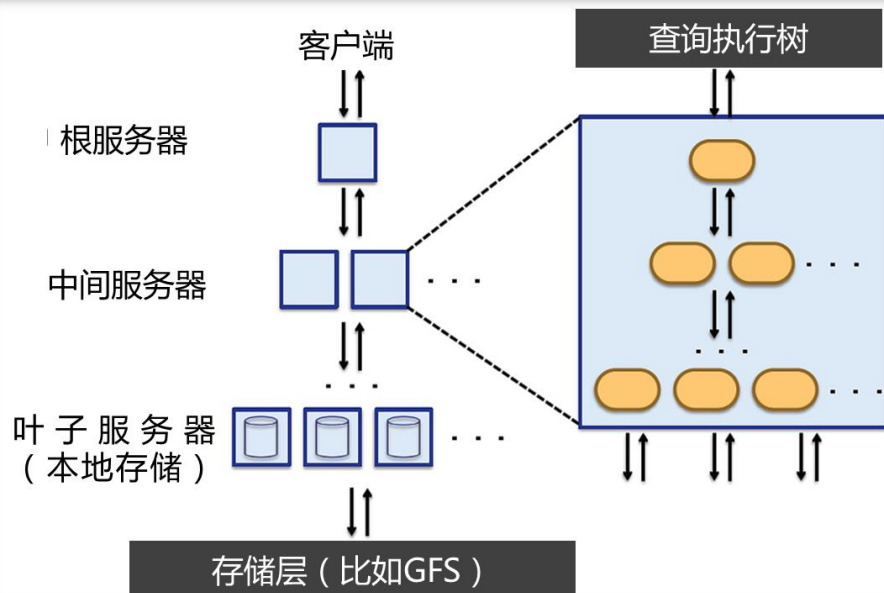
• 2.7.4 查询语言与执行

- ✓ 接下来的第2层直到叶子节点都将进行类似的查询重写。
- ✓ 在每个部分得到子查询结果之后，会依次从下到上执行一个查询结果的聚集过程，直到得到最后的结果并返回给用户。

```
DocId: 10 r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20 r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```



2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

Dremel是一个多用户系统，因此，同一时刻往往会有多个用户进行查询。

为了维护系统的性能稳定，需要查询分发器（query dispatcher）来进行负载均衡。

查询分发器还负责系统的容错，当某个节点的执行时间过长，会分配其他节点来执行该任务。执行时间信息由查询分发器维护一个直方图来体现。

2.7 海量数据的交互式分析工具Dremel

• 2.7.4 查询语言与执行

查询分发器有一个很重要的参数，它表示在返回结果之前，一定要扫描百分之多少的tablet。

Google的使用经验表明，设置这个参数到较小的值，通常能显著地提升执行速度，例如98%（而不是100%），即不扫描完所有数据。

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

► 2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 2.7.5 性能分析

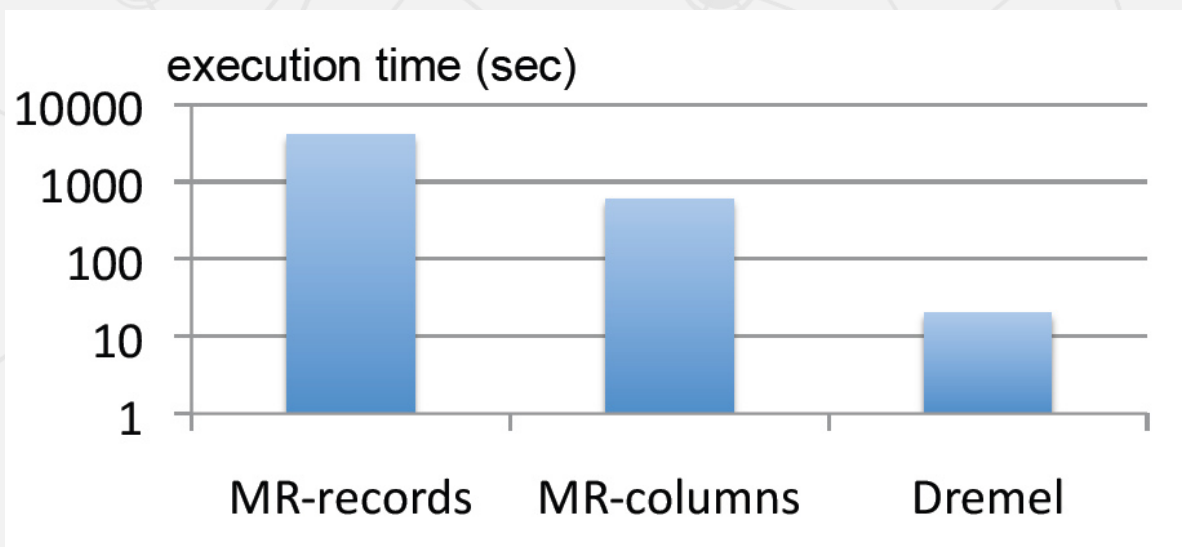
由于Dremel并不开源，我们只能通过Google论文中的分析大致了解其性能。Google的实验数据集规模如下表：

表名	记录数 (亿)	规模 (未压缩, TB)	域数目	数据中心	复制因子
T1	850	87	270	A	3
T2	240	13	530	A	3
T3	40	70	1200	A	3
T4	>10000	105	50	B	3
T5	>10000	20	50	B	2

2.7 海量数据的交互式分析工具Dremel

• 2.7.5 性能分析

下图展现了2个MR（MapReduce）任务和Dremel的执行耗时。所有的任务均运行在3000个工作节点上。

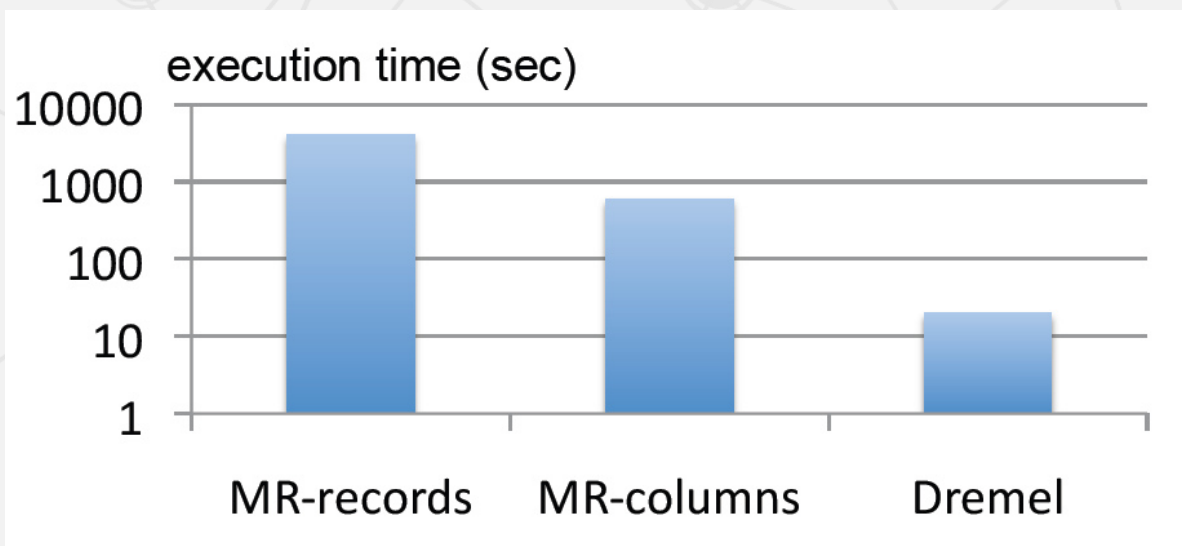


实验中，Dremel和MR-列状存储实际仅读取大约**0.5TB**的压缩列状数据，而MR-面向记录存储则读取**87TB**数据（即全部数据）。

2.7 海量数据的交互式分析工具Dremel

• 2.7.5 性能分析

MR从面向记录转换到列状存储后，性能提升了一个数量级（从小时到分钟），而使用Dremel则又提升了一个数量级（从分钟到秒）。



2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

► 2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

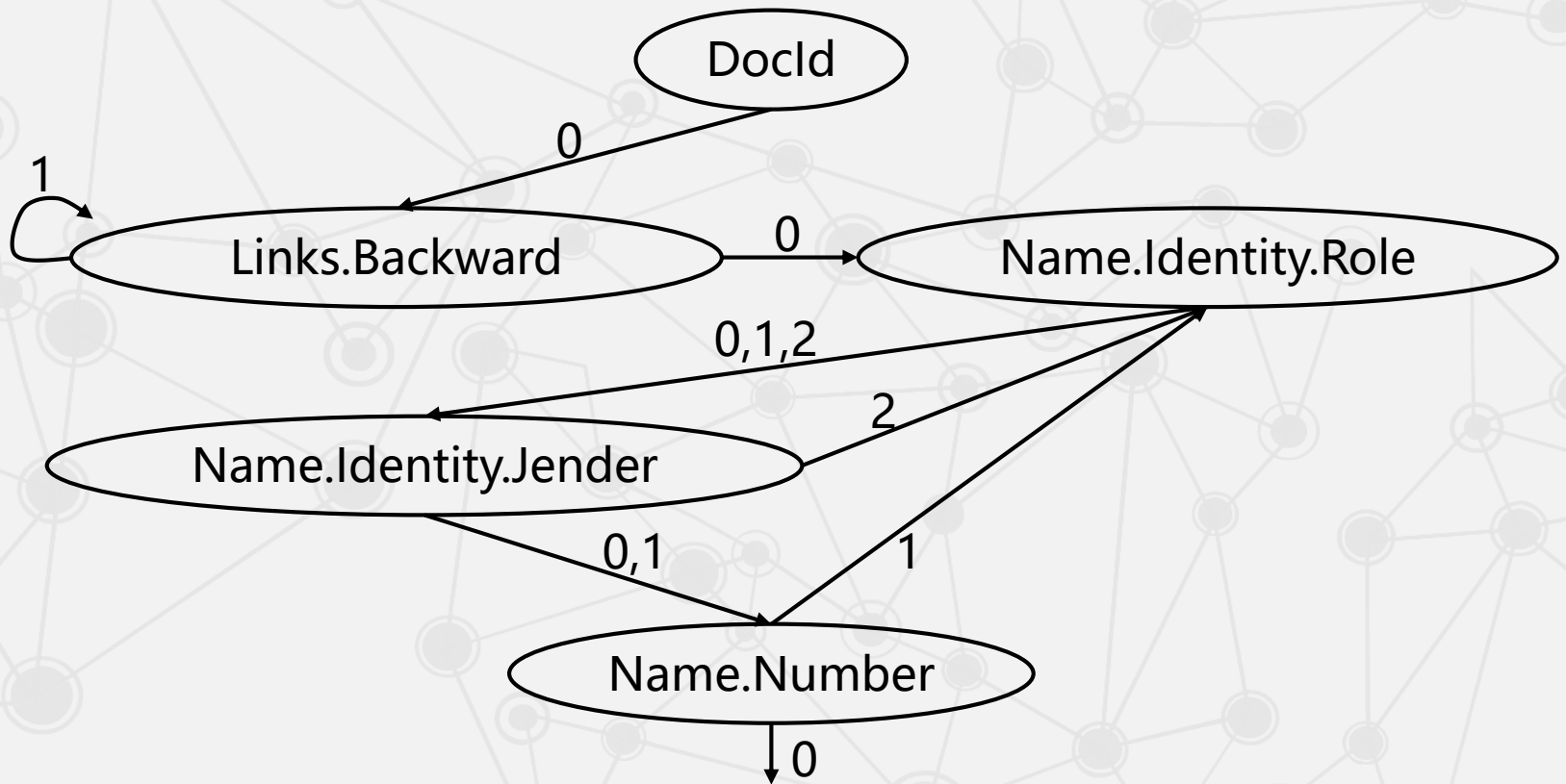
● 小结

1

Dremel和MapReduce并不是互相替代，而是相互补充的技术。在不同的应用场景下各有其用武之地。

2

Drill的设计目标就是复制一个开源的Dremel，但是从目前来看，该项目无论是进展还是影响力都达不到Hadoop（复制MapReduce）的高度。



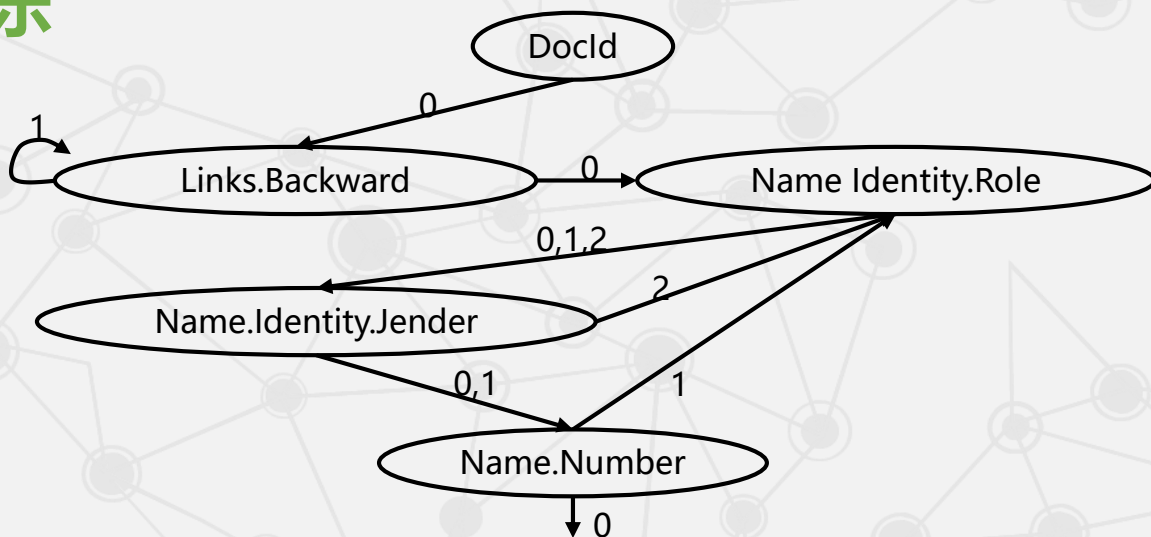
2.7 海量数据的交互式分析工具Dremel

• 1. 数据结构的无损表示

DocId		
value	r	d
11	0	0

Links.Backward		
value	r	d
10	0	2
30	1	2

Name.Number		
value	r	d
2023112009	0	2
2021225883	1	2
Null	1	1



Name.Identity.Role		
value	r	d
teacher	0	2
student	2	2
Null	1	1
student	1	2

Name.Identity.Jender		
value	r	d
female	0	3
Null	2	2
Null	1	1
male	1	3

[illegible]

序号	当前FSM	写入值	下一个重复深度值	动作
1	DocId (开始)	11	0	跳转至Links.Backward
2	Links.Backward	10	1	跳转至Links.Backward
3	Links. Backward	30	0	跳转至Name.Identity.Role
4	Name.Identity.Role	teacher	2	跳转至Name.Identity.Jender
5	Name.Identity.Jender	female	2	跳转至Name.Identity.Role
6	Name.Identity.Role	student	1	跳转至Name.Identity.Jender
7	Name.Identity.Jender	Null	1	跳转至Name.Number
8	Name.Number	2023112009	1	跳转至Name.Identity.Role
9	Name.Identity.Role	Null	1	跳转至Name.Identity.Jender
10	Name.Identity.Jender	Null	1	跳转至Name.Number
11	Name.Number	2021225883	1	跳转至Name.Identity.Role
12	Name.Identity.Role	student	0	跳转至Name.Identity.Jender
13	Name.Identity.Jender	male	0	跳转至Name.Number
14	Name.Number	Null	0	结束

序号	当前FSM	写入值	下一个重复深度值	动作
1	DocId (开始)	11	DocId: 11 Links Backward: 10 Backward: 30 Name Identity Role: 'teacher' Jender: 'female' Identity Role: 'student' Number: '2023112009' Name Number: '2021225883' Name Identity: Role: 'student' Jender: 'male'	rd
2	Links.Backward	10		rd
3	Links. Backward	30		Role
4	Name.Identity.Role	teacher		ender
5	Name.Identity.Jender	female		Role
6	Name.Identity.Role	student		ender
7	Name.Identity.Jender	Null		er
8	Name.Number	2023112009		Role
9	Name.Identity.Role	Null		ender
10	Name.Identity.Jender	Null		er
11	Name.Number	2021225883		Role
12	Name.Identity.Role	student		ender
13	Name.Identity.Jender	male		er
14	Name.Number	null		结束